

选题	2024 年第十四届 APMCM 亚太地区大学生数学建模竞赛（中文赛项）	参赛编号
B		apmcm2410053 6

基于关键特征集成学习的洪水灾害数据分析与预测研究

摘要

针对问题 1，首先计算出所给数据季风强度、地形排水等基础指标和洪水概率的最大值、最小值、平均值、中位数等基本统计量，完成数据预处理，随后对各个指标及洪水概率进行可视化分析，并计算上述所有指标分布的偏度与峰度系数，结果可得**训练集和测试集中各指标分布均呈现轻微的正偏态(右偏)和尖峰厚尾，训练集洪水概率分布呈现正态分布**。随后，引入 Spearman 相关系数进相关性分析，并加以假设检验，**发现各指标之间呈现独立性，同时均与洪水概率呈现有层次的正相关关系**，随后取相关性平均值以上的指标为高相关性指标，并对其进行相应分析，提出相关建议与措施。

针对问题 2，首先标准化洪水概率数据，利用 **K-Means++ 聚类算法**将其聚类为高、中、低风险三大类，随后对聚类结果进行可视化分析，基于簇进行数据表分组，并计算每个分组的最大、最小值，**验证数据完整聚类为三簇无交错类别**，计算三组数据表指标统计量，得出**指标数值与风险高低呈现正相关性，且不同指标在同一风险类别中呈现相似的数据分布情况**。评价问题定义为回归+分类，首先基于分组数据中最大、最小值构造回归数值与分类类别之间的映射区间，并训练**逻辑回归分类模型**实现映射区间的缺漏填充，成功构建完整的映射区间，随后，基于原始 20 指标构建**线性回归模型**作为**基准模型**，**R2 分数 0.84**，三分类准确率**0.76**，得出指标与洪水概率之间存在**正线性相关性**，并对所有指标之和与洪水概率进行可视化确认其间呈**强正线性相关性**，引入指标和作为关键引导指标，然后将原始指标在**样本维度升序排列**，构造排序指标，以消除指标维度数据差异性，最后，将**排序指标与指标和**作为最终指标训练 **CatBoost 回归模型**实现了**优于基准模型的效果**，**R2 分数 0.87**，并且其在三分类任务上同样表现优秀，三分类准确率**0.76**，同时，利用 **K 折交叉验证法**验证了模型的**低灵敏度与高泛化性**。

针对问题 3，首先基于问题 1 中各指标与洪水概率相关性选出**相关性大于平均值的指标**，加入**指标和**作为关键指标，训练 CatBoost 回归模型，得出较优回归预测模型。随后，使用 **PCA 主成分分析**进行数据降维，将 20 维指标降至**4 维关键指标**，同时依旧加入**指标和**组成 5 个指标，用以训练 CatBoost 回归模型，得出**性能远胜基准模型的最优回归预测模型**，**R2 分数 0.97**，同时作为佐证，求出其**三分类准确率 0.91**。

针对问题 4，直接选用问题 3 中得出的最优 CatBoost 回归模型进行预测，填写预测结果，并对结果使用直方图、折线图进行可视化分析，计算其偏度、峰度系数，得出**预测结果服从正态分布**，并与其他性能较低回归模型进行对比，验证其预测结果正态性与合理性。

关键词：CatBoost 回归模型 线性回归模型 逻辑回归分类模型 连续映射区间构造 K-Means++ 聚类算法 PCA 主成分分析 K 折交叉验证

一、问题重述

1.1 问题背景

洪水作为一种严重的自然灾害，对人类社会和生态环境造成了巨大的影响。历史上，洪水灾害的发生往往与自然因素如暴雨、融冰化雪、风暴潮等有关。然而，随着人口的增长和人类活动的增加，如乱砍滥伐、围湖造田等，这些活动加剧了地表状态的改变，进而影响了汇流条件和洪水灾害的程度。近年来，全球洪水灾害频发，造成了巨大的经济损失和人员伤亡。因此，对洪水灾害进行数据分析与预测，对于提前预警和减少灾害损失具有重要意义。

1.2 问题要求

附件数据给出了各地季风强度、地形排水、河流管理等基本信息，以及该地发生洪水的概率。为了能更好地根据已知信息预测洪水的发生概率，最小化灾害损失，现需结合实际情况与所给信息建立数学模型，分析以下问题：

问题 1：依据附件 1 中给出的基本信息，可视化处理相关数据，并单独分析洪水发生概率与每个基本信息的相关性，判断其关联程度大小，给出相关结论和建议。

问题 2：将 `train.csv` 中洪水发生的概率聚类成不同风险类别，分析高、中、低风险洪水事件的指标特征。选取合适的指标，计算不同指标的权重，建立预警评价模型，并进行模型的灵敏度分析。

问题 3：基于问题 1 中指标分析的结果，建立洪水发生概率的预测模型。从 20 个指标中选取合适指标，预测洪水发生的概率，并验证模型的准确性。探讨如果仅用 5 个关键指标，如何调整改进预测模型。

问题 4：利用问题 2 中建立的洪水发生概率预测模型，预测附件 `test.csv` 中所有事件的洪水发生概率，并将结果填入 `submit.csv`。绘制预测概率的直方图和折线图，分析其分布特征，判断是否服从正态分布。

附件：

1. `train.csv` - 包含超过 100 万条洪水数据，涵盖洪水事件 id 和 20 个指标得分，以及发生洪水的概率。
2. `test.csv` - 包含超过 70 万条洪水数据，涵盖洪水事件 id 和 20 个指标得分，缺少发生洪水的概率。
3. `submit.csv` - 包含 `test.csv` 中的洪水事件 id，缺少发生洪水的概率。

二、问题分析

2.1 问题 1 的分析

针对问题 1，首先对 `train.csv` 中所给数据进行最大值、最小值、平均值、中位数等基本统计量计算、数据预处理等基础性工作，并记录洪水概率的最大、最小值，基于这些数据，采用概率密度直方图、核密度估计（Kernel Density Estimation, KDE）图可视化洪水概率的分布情况，同时用频数统计直方图可视化 `train.csv` 与 `test.csv` 中季风强度、地形排水等各个指标特征的分布情况，并计算上述所有指标分布的偏度与峰度系数用以量化各项评估指标与洪水概率的分布

情况。随后，引入 Spearman 相关系数进相关性分析，并加以假设检验，用以量化各项指标与洪水概率之间的关系。最后，综合以上获得的相关性、数据分布情况等分析结果进行统计分析，指出对洪水概率影响较大的指标，并据此提出相应的预防措施。

2.2 问题 2 的分析

针对问题 2，首先标准化 train.csv 中洪水概率数据，并利用 K-Means++ 聚类算法将其聚类为三簇，随后对聚类结果进行箱型图、频数分布直方图可视化，基于簇进行数据表分组，并计算每个分组的最大、最小值，用以评估聚类结果是否存在交错，最后划分风险为高、中、低三类，将其作为分类标签，并对其分别进行统计分析可视化，用以分析不同风险对应指标特征。风险预警评价问题实质是基于连续数值到逻辑类别区间映射的回归+分类的合并问题，首先基于先前得出的三类分组数据中最大、最小值构造连续回归数值转离散类别数值的映射，并训练逻辑回归模型建立符合分布的洪水概率到风险等级的连续映射，实现有空缺映射区间的填充工作，以保证最终回归+分类模型评价能力的鲁棒性，随后，基于所有指标构建线性回归模型作为基准模型，实现较优结果，探索指标特征与洪水概率间潜在的线性相关性，并对所有指标之和与洪水概率进行可视化以确认其线性相关性，因此引入指标和作为关键指标，然后将季风强度、地形排水等指标在样本维度上从小到大排序，构造排序指标，从而消除指标维度上的数据差异性，最后，将排序指标与指标和作为最终指标训练 CatBoost 模型实现了优于基准模型的效果，并且其在三分类评价任务上同样表现优秀，同时，利用 K 折交叉验证法验证了模型的低灵敏度与高泛化性。

2.3 问题 3 的分析

针对问题 3，首先基于问题 1 分析出的各指标与洪水概率的 Spearman 相关系数选出相关性较高的指标特征作为基本指标，并加入指标和作为关键指标，训练 CatBoost 回归模型，得出较优回归预测模型。随后，使用主成分分析（Principle Component Analysis, PCA）进行数据降维，将 20 维指标特征降至 4 维作为关键指标，同时依旧加入指标和组成 5 个指标，用以训练 CatBoost 回归模型，得出最优回归预测模型。

2.4 问题 4 的分析

针对问题 4，直接选用问题 3 中得出的最优 CatBoost 回归模型进行 test.csv 的特征工程，才用最优 5 个关键指标进行结果预测，于 submit.csv 中填写预测结果，并对预测结果使用直方图、折线图（随机采样 1000 样本）进行可视化分析，计算其偏度峰度系数，分析其分布正态性，并与其他两个性能较低的 CatBoost 回归模型进行分布对比，验证其预测结果正态性与合理性。

文章总体思路如图 1 所示：

回归+分类 合并问题

回归为主，分类为辅的预测问题

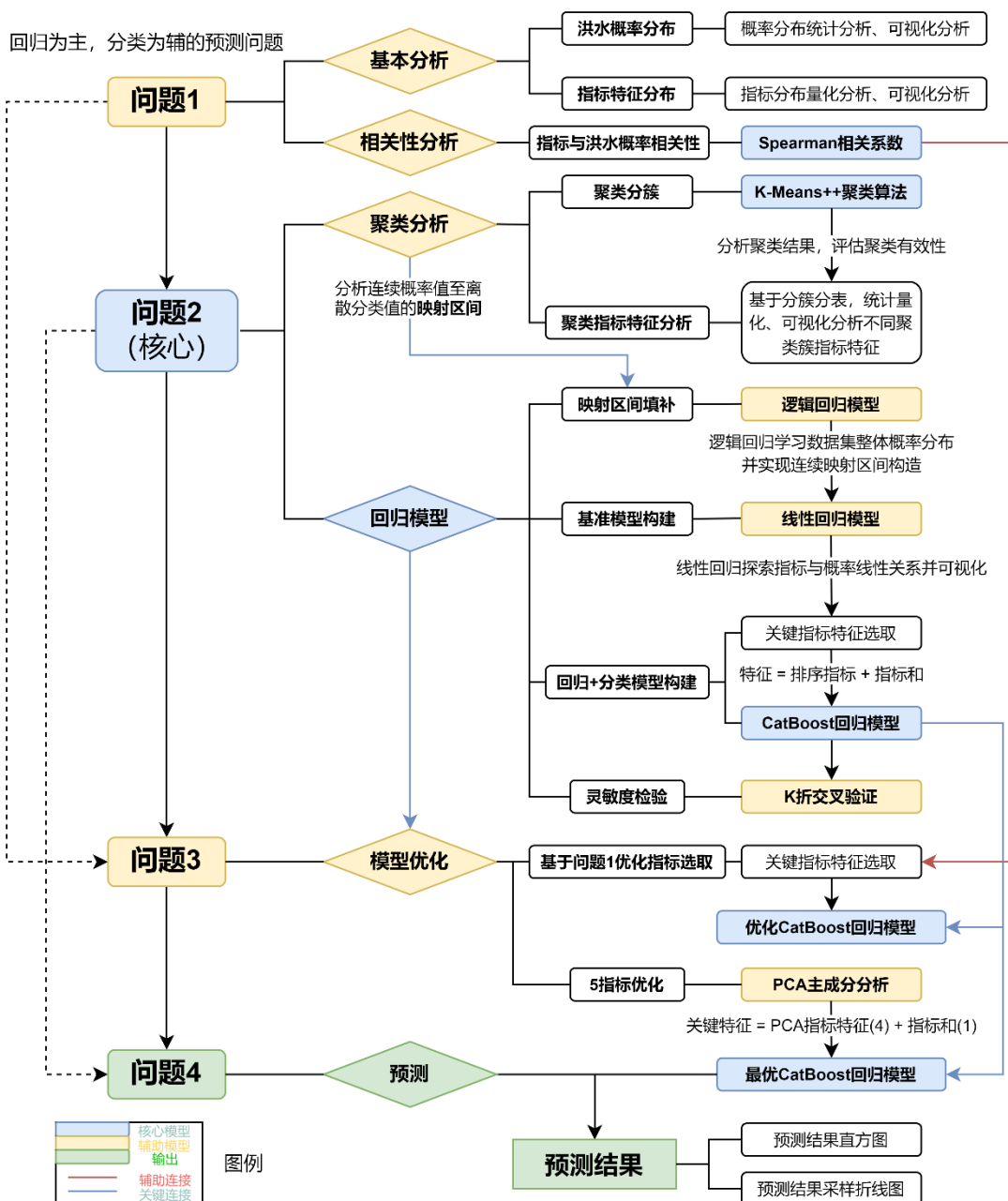


图 1 总体思路图

三、模型假设

根据上述问题分析，我们做出如下假设：

1. 假设洪水发生的概率和影响因素在短期内相对稳定，即考虑的时间范围内没有显著的气候或人为变化。。
2. 假设使用的历史数据或观测数据是准确和可靠的，能够充分反映洪水发生的情况和影响因素的变化。

四、符号说明

符号	说明
K_u	峰度系数
S_u	偏度系数
ρ	Spearman 相关系数
$d(x_i, c_j)$	x_i 与 c_j 之间的欧氏距离
ω_i	权重系数
x_i	样本点 x 的第 i 个分量
L	损失函数
$\hat{y}$	预测值
Org-20	使用最初的 20 个指标
Sorted-20	使用 20 个指标的排序指标
Sum-1	使用 20 个指标的和作为指标
Selected-10	使用精选的 10 个指标
PCA-4	使用 PCA 后得到的 4 个最主要的成分作为指标

五、问题 1 模型的建立与求解

5.1 基于数据的数字特征进行描述

题目要求对数据进行可视化处理,为了更好地描述洪水发生概率分布、基本信息特征的分布规律,本文引入峰度系数、偏度系数来描述统计数据。

(1) 峰度系数

峰度系数 (Kurtosis) 用于衡量数据分布的峰度程度。例如 $K_u = 0$ 时,数据的分布为正态分布; $K_u < 0$ 时,数据分布的峰度较小,数据更分散; $K_u > 0$ 时,数据分布的峰度较大,数据更集中。并且 $|K_u|$ 越大,数据分布的图像更加“陡峭”。峰度系数 K_u 的计算公式如下:

$$K_u = E\left[\left(\frac{X - \mu}{\sigma}\right)^4\right] = \left(\frac{\mu}{\sigma}\right)^4$$

(2) 偏度系数

偏度系数 (Skewness) 用于衡量数据分布的偏斜程度。例如 $S_k = 0$ 时,数据的分布为正态分布; $S_k < 0$ 时,数据呈左偏分布; $S_k > 0$ 时,数据呈右偏分布。并且 $|S_k|$ 越大,数据分布的偏斜越明显。峰度系数 S_k 的计算公式如下:

$$S_k = E\left[\left(\frac{X - \mu}{\sigma}\right)^3\right] = \left(\frac{\mu}{\sigma}\right)^3$$

5.2 数据可视化

首先对原始数据进行可视化分析,作出洪水发生概率密度图像与核密度估计图像,并在其左上角展示分布的偏度峰度,图像如下:

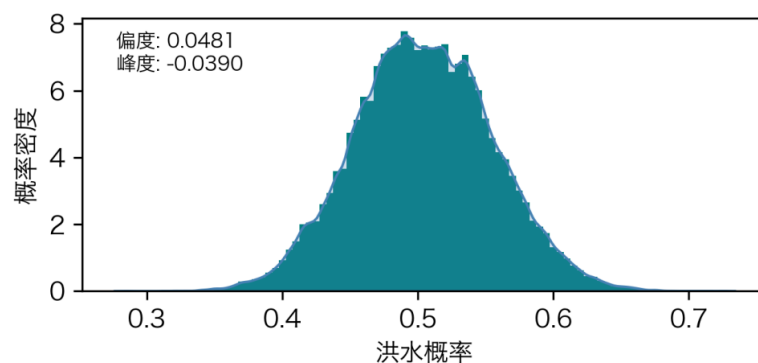


图 2 洪水概率密度图

可见洪水概率分布基本符合正态分布。

基本指标的分布直方图如下：

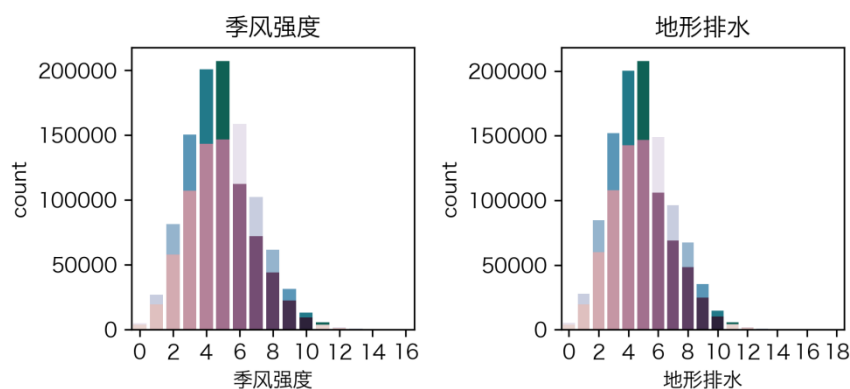


图 3 部分指标柱状图

图中紫红色为测试集分布情况，蓝绿色为训练集分布情况此处仅采样季风强度与地形排水的指标分布情况进行可视化，其余指标的分布情况基本相同，且均呈现出训练、测试集服从基本相同的分布且训练集数据量比测试集大 40%左右。

下面对已有数据定量分析，统计训练集、测试集中数据分布的偏度与峰度系数。

表 1 数据集指标偏、峰度统计量

	训练集偏度系数	训练集峰度系数	测试集偏度系数	测试集峰度系数
平均值	0.441813837	0.247223868	0.44093833	0.247306689
标准差	0.011392705	0.035052238	0.012480602	0.034875173
最小值	0.419867945	0.187350475	0.413857617	0.182297125
25%	0.436629875	0.232680673	0.435625489	0.228729943
50%	0.441398792	0.243670623	0.442318853	0.246637074
75%	0.449001103	0.262154138	0.449952157	0.263493856
最大值	0.464098419	0.339472861	0.460378327	0.315153435

基于上表定量数据对整个数据集的分析如下：

1. 偏度系数：平均偏度系数接近 0.44，表明数据分布呈现轻度的正偏态（右偏），即大部分数据分布在均值左侧，尾部向右延伸。其标准差较小，说明各个指标的偏度相对一致。最小值和 25%分位数显示有些指标的偏度较低，而 75%分位数和最大值则显示少数指标有较高的正偏度。

2. 峰度系数：平均峰度系数在 0.24 到 0.25 之间，略高于 0（正态分布的峰

度系数为 3)，表明数据分布相对于正态分布有轻微的尖峰厚尾特性。峰度的标准差相对较大，说明不同指标的峰度差异较大。

3. 分布一致性：训练集和测试集的偏度和峰度系数非常接近，这表明两个数据集在分布形状上具有较高的一致性。

4. 数据范围：最小值和最大值展示了数据分布的整个范围，可以看出指标值在 0.4 到 0.46 之间变化，这可能表明数据的变异性有限。

5. 中位数（50%分位数）：中位数接近平均值，进一步确认了数据分布的正偏态。

6. 四分位数间距（IQR）：通过比较 25%和 75%分位数，可以观察到数据的分散程度。IQR 较小，表明大部分数据点集中在中位数附近。

7. 异常值：由于偏度和峰度系数的值并不极端，可能没有明显的异常值，但最大值和最小值的指标可能需要进一步检查是否有异常值。

整体而言，数据集的分布呈现轻微的正偏态(右偏)和轻微的尖峰厚尾，但整体分布相对集中。

5.3 利用 Spearman 相关系数判断指标与洪水概率之间的相关性

Spearman 相关系数是一种非参数的相关性度量，可以等级化变量之间相关性，并且不局限于线性关系，适用于非线性关系的评估，其计算公式如下：

$$\rho = \frac{\sum_{i=1}^N (R_i - \bar{R})(S_i - \bar{S})}{[\sum_{i=1}^N (R_i - \bar{R})^2 \sum_{i=1}^N (S_i - \bar{S})^2]^{\frac{1}{2}}} = 1 - \frac{6 \sum d_i^2}{N(N^2 - 1)}$$

其中， R_i 与 S_i 均为对应观测值第 i 个取值的等级， $[\bar{R}]$ 与 $[\bar{S}]$ 均为对应观测值取值的平均等级， N 为观测值的总数量， $d_i = R_i - S_i$ 。

Spearman 相关系数评估了两变量之间的单调关系，即两个变量同时增加（或减少）时相关系数趋近于 1（或-1）；两个变量的变化之间未出现明显关系时趋近于 0。

5.4 Spearman 相关系数的检验

本文对 20 个指标采取 Spearman 相关系数检验，检验步骤如下：

(1)提出假设

原假设 H_0 : Spearman 系数 $\rho \neq 0$

备择假设 H_1 : Spearman 系数 $\rho = 0$

设定置信水平为 99.5%

(2)计算 P 值

本文采用 Python 中 statsmodels 中 multipletests 进行 Spearman 相关系数检验，得出所有指标之间以及所有指标与洪水概率之间的修正后 Spearman 相关系数。将结果拆分为两个表，分别展示指标之间以及指标与洪水概率之间的 Spearman 相关系数：

指标之间 Spearman 相关系数表如下（由于大小限制详情见附表）：

表 2 Spearman 相关系数检验的 P 值					
	季风强度	地形排水	河流管理	...	政策因素
季风强度	1.0000	0.0000	0.0000	...	0.0000
地形排水	0.0000	1.0000	0.0000	...	0.0000
河流管理	0.0000	0.0000	1.0000	...	0.0000

⋮	⋮	⋮	⋮	⋮	⋮
政策因素	0.0000	0.0000	0.0000	0.0000	1.0000

表 2 是 Spearman 相关系数检验的 P 值构成的矩阵，其对角线元素为 1，其余元素均为 0，即其为单位矩阵，可知任意两指标之间的 P 值均为 0，故接受原假设，认为 Spearman 系数 $\rho\neq0$ 。

将其可视化热力图如下：

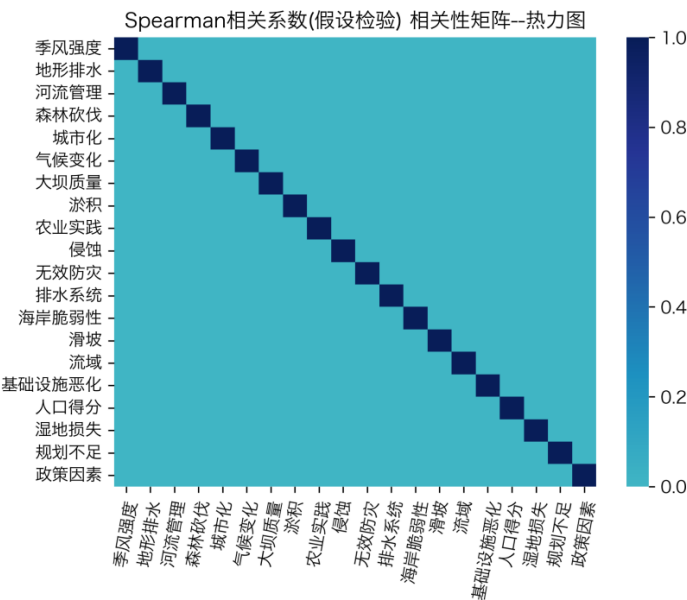


图 4 Spearman 相关系数（假设检验）相关性矩阵

从上图可直观看出指标数据之间呈现出**完全独立性**。

假设检验之后指标与洪水概率之间 Spearman 相关系数：

由上表可知：

表 3 Spearman 相关系数				
季风强度	地形排水	河流管理	森林砍伐	城市化
0.18028	0.18048	0.17893	0.17701	0.17265
气候变化	大坝质量	淤积	农业实践	侵蚀
0.17712	0.17946	0.17874	0.17514	0.17143
无效防灾	排水系统	海岸脆弱性	滑坡	流域
0.17597	0.17044	0.17029	0.17664	0.17464
基础设施恶化	人口得分	湿地损失	规划不足	政策因素
0.18140	0.17799	0.17514	0.17325	0.17387

1. 基础设施恶化对洪水的影响最大，海岸脆弱性对洪水的影响最小。
 2. 20 个指标与洪水概率的 Spearman 相关系数都在 0.175 左右，说明 20 个指标均与洪水发生概率呈正相关，但是相关性均较弱。
- 通过计算统计量与洪水概率的 Spearman 相关系数，我们得到各统计量与洪水发生概率的相关性如下：

表 4 统计量与洪水发生概率的 Spearman 相关系数			
统计量	平均值	最小值	最大值
ρ	0.176044	0.170289	0.181399
统计量	25%	50%	75%
ρ	0.173718	0.176304	0.178786

由上表可知，平均值作为统计量，与洪水发生概率的相关性居中，因此选择平均值作为划分指标来进行模型优化。因此选取出检验后 Spearman 相关系数高于平均的指标如下共 10 个：

表 5 精选指标				
基础设施恶化	地形排水	季风强度	大坝质量	河流管理
淤积	人口得分	气候变化	森林砍伐	滑坡

(1) 可能的原因

强烈的季风和降雨可能导致洪水，森林砍伐和城市化造成水土流失增加了洪水的风险，气候变化导致极端天气事件增多，洪水风险上升，基础设施如排水系统和大坝质量问题，减少了防洪能力

(2) 建议与措施

- **提升监测预警：**加强洪水监测和预警系统。
- **合理规划土地：**避免在洪水易发区进行开发。
- **增加绿化：**提高森林覆盖率，减少水土流失。
- **改善水利设施：**加强河流管理和排水系统建设。
- **公众教育：**提高公众防洪意识和自救能力。
- **制定防洪措施：**包括风险评估和应急计划。
- **跨部门协作：**确保防洪措施得到有效执行。
- **应用科技：**使用现代技术提高洪水预测和管理效率。

六、问题 2 模型的建立与求解

6.1 低，中，高风险聚类

6.1.1.K-Means++聚类过程

K-Means++是 K-Means 聚类算法的一种改进版本，它通过智能地选择初始聚类中心来提高 K-Means 的性能(李航 2022： 216-227)。K-Means++的目标是更好地选择初始聚类中心，以减少算法的迭代次数，提高聚类的质量。具体步骤如下：

1. **选择第一个聚类中心：**随机选择一个数据点作为第一个聚类中心。
2. **选择后续聚类中心：**对于每个数据点，计算它与已选择的聚类中心之间的最短距离 $D(x)$ （即与最近的聚类中心之间的距离）。每个数据点作为下一个聚类中心的概率为：

$$p(x) = \frac{D(x)^2}{\sum_{x \in X} D(x)^2}$$

其中 x 代表任意一个数据点， X 表示数据集。这样保证了距离更远的数据点更有可能被选为下一个聚类中心，以确保新的聚类中心较好地覆盖数据分布。

3. **完成聚类中心的选择：**重复步骤 2 直到选择足够数量的聚类中心（一般为 k 个，我们称将数据分成了 k 个簇）。
4. **计算每个数据点到聚类中心的距离：**对于每个数据点，计算它与每个聚类中心的距离，通常使用欧氏距离或其他距离度量方式。对于数据点与聚类中心的距离由以下公式给出：

$$d(x_i, c_j) = \sqrt{\sum_{k=1}^N (x_{ik} - c_{jk})^2}$$

其中 x_i 为第 i 个样本点， c_j 为第 j 个聚类中心， x_{ik} 是第 i 个样本点的第 k 个分量， c_{jk} 是第 j 个聚类中心的第 k 个分量。

5. **分配数据点到最近的聚类中心：**将每个数据点分配到距离它最近的聚类中心所属的簇。
6. **更新聚类中心：**重新计算每个簇的中心，即该簇所有数据点的平均值。将该平均值作为新的聚类中心。即新的中心为：

$$c'_j = \frac{1}{N_{X_j}} \sum_{x_i \in X_j} x_i$$

其中 c'_j 为新的聚类中心，其中 X_j 为以 c_j 为聚类中心的聚类， N_{X_j} 为 X_j 中数据点的个数。

7. **完成聚类：**重复步骤 4，5，6，直到聚类中心不再改变，或达到预定义的停止条件（比如达到最大迭代次数）。

最终得到每个数据点所属的簇，即完成了 K-Means++ 聚类。

使用 Python 进行求解，再用箱型图和直方图进行可视化，得到以下结果：

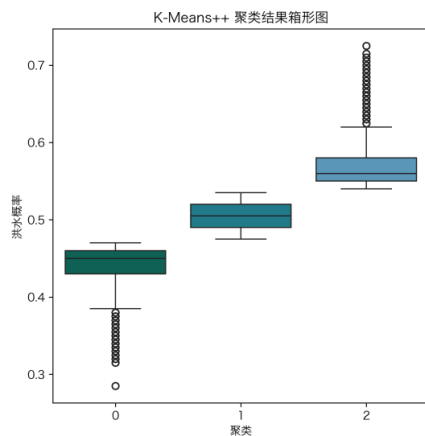


图 5 K-Means++ 聚类结果（箱型图）

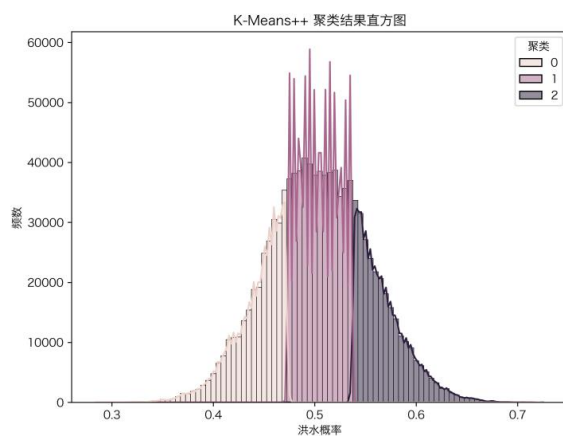


图 6 K-Means++ 聚类结果（直方图）

从图中不难看出，低，中，高风险的洪水概率是基本无交错的且大致服从正态分布，但仍旧需要精确的数据来证明聚类结果是有效的。

基于聚类结果对总表进行分表，并合并统计区间分布情况如下：

表 6 低, 中, 高风险洪水概率分布情况						
	样本数	平均值	最小值	25%	75%	最大值
低风险	282202	0.4423	0.285	0.43	0.46	0.47
中风险	492938	0.5046	0.475	0.49	0.52	0.535
高风险	273435	0.5684	0.54	0.55	0.58	0.725

从表中可见, 不同风险类别情况下, 三个风险等级对应已有区间分别为 [0.285,0.47], [0.475,0.535], [0.54,0.725], 因此我们判断 K-Means++ 完整将数据聚类到了三个无交错簇中。

至此, 我们将所有 train.csv 中所有数据根据洪水发生的概率划分成了低, 中, 高风险三个类别。

但是对于风险评级的划分, 我们希望得到 [0,1] 区间上连续的三个子区间, 而事实上得到的三个区间分别为 [0.285,0.47], [0.475,0.535], [0.54,0.725], 显然这些区间是存在空缺的, 为了填补空缺实现一个完整的回归区间划分使其较好地满足回归+三分类模型需求, 下面基于对 train.csv 中洪水概率的统计分析, 使用逻辑回归对其进行区间填补。

6.1.2 逻辑回归填补区间

逻辑回归, 即对数概率回归, 是一种线性模型, 是一种用于分类问题的算法(李航 2022: 77-93)。通过学习特征与类别之间的关系, 逻辑回归可以预测新数据点属于哪个类别, 输出类别的概率, 其过程如下:

1. **收集数据:** 收集具有标签的训练数据集, 每个数据点包括特征值和类别标签。
2. **特征工程:** 对数据进行特征提取和选择, 包括数据清洗、特征缩放、特征选择等操作。
3. **定义预测函数:** 假设我们的模型可以用如下形式表达:

$$h_{\theta}(x) = g(\theta^T x)$$

其中 $g(z)$ 是 Sigmoid 函数, 定义为:

$$g(z) = \frac{1}{1 + e^{-z}}$$

$g(z)$ 的导数满足 $[g'(z) = g(z)(1 - g(z))]$ 。

则最终预测函数表达式为:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

其中 $\theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$, $\theta_i (i = 1, 2, 3, \dots, n)$ 为 x_i 对应权重, 需要后续过程计算得出。

最后样本为正样本的概率为 $h_{\theta}(x)$, 样本为负样本的概率为 $1 - h_{\theta}(x)$ 。

4. **定义损失函数:** 我们使用交叉熵损失函数来度量模型预测值与真实标签之间的误差 (由于篇幅原因, 我们不给出推导过程):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

5. **最小化损失函数:** 通过梯度下降等优化算法, 最小化损失函数, 得到最佳的参数 θ 。
6. **预测:** 当模型训练完成后, 我们可以用训练好的参数对新数据进行分类预测。分类规则为: 如果正样本的概率大于负样本的概率, 则样本被判定为正样本, 否则被判定为负样本, 这等价于

$$\frac{h(x)}{1-h(x)} > 1$$

首先将训练集给出的洪水概率作为特征，将聚类出的类别作为分类标签，训练**逻辑回归分类器**，原始的区间[0.2825, 0.7275]微分切片为 1048575 份，为了提升精确度，将其提升 100 倍，即 100×1048575 份，生成一个相对连续的概率区间划分序列，随后将该序列用先前训练出的逻辑回归分类模型进行分类，成功找到三个基本连续的区间[0.282500, 0.472481], [0.472481, 0.537549], [0.537549, 0.727500]。因此，得出区间划分的结论为：

$$\left\{ \begin{array}{l} \text{低风险区间: } [0, 0.472481) \\ \text{中风险区间: } [0.472481, 0.537549) \\ \text{高风险区间: } [0.537549, 1] \end{array} \right.$$

至此，连续的回归数值到离散的类别数值的映射区间构造完成。

6.3 分析具有高、中、低风险的洪水事件的指标特征

对高、中、低风险的洪水事件进行统计分析以及可视化，得出如下结果（以低风险为例，其余见附表）：

表 7 低风险区的各指标分布情况					
	季风强度	地形排水	河流管理	森林砍伐	城市化
平均值	4.489	4.474	4.518	4.514	4.520
最小值	0	0	0	0	0
最大值	16	18	16	16	16
	气候变化	大坝质量	淤积	农业实践	侵蚀
平均值	4.511	4.517	4.491	4.518	4.517
最小值	0	0	0	0	0
最大值	17	16	16	16	17
	无效防灾	排水系统	海岸脆弱性	滑坡	流域
平均值	4.503	4.532	4.535	4.496	4.491
最小值	0	0	0	0	0
最大值	16	17	17	16	16
	基础设施恶化	人口得分	湿地损失	规划不足	政策因素
平均值	4.492	4.490	4.523	4.512	4.513
最小值	0	0	0	0	0
最大值	16	16	16	16	16

从表中可见，低风险情况下的各大指标平均数值差异不大，而该情况在其他两种风险类别中也及其相似，可得出相应结论，即**同类风险程度下各项指标数值分布极其相似**。

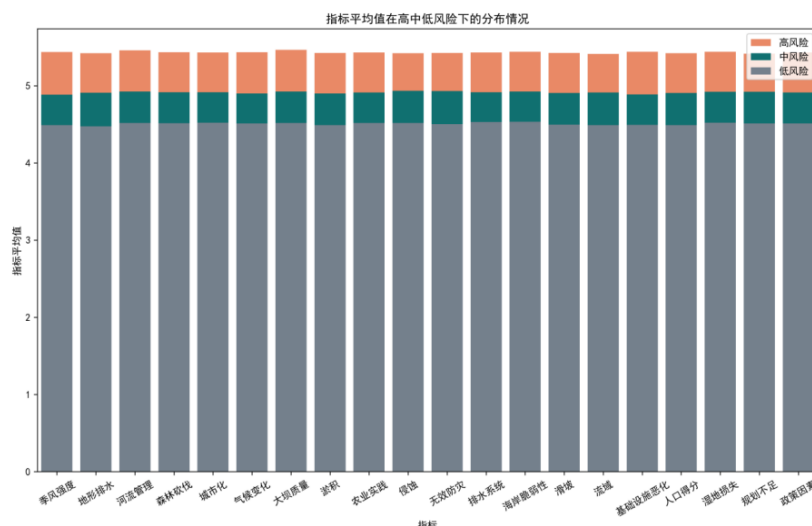


图7 指标平均值在高中低风险下单分布情况

上图可视化结果印证了同类风险程度下各项指标数值分布极其相似的数据分析结论，接下来我们通过线性回归建立预警评价模型。

6.4 建立发生洪水不同风险的预警评价模型

6.4.1 选取评价指标

为了评估我们最后得到的回归+分类合并模型，我们先引入 **R² 分数**和**三分类准确率**。

- **R² 分数**：也称为确定系数（Coefficient of Determination），是衡量回归模型拟合优度的一个常用指标,其计算公式如下：

$$R^2 = \frac{\sum(\hat{y}_i - \bar{y})^2}{\sum(y_i - \bar{y})^2}$$

它直观地反映了模型预测能力的强弱，并且可以用来解释模型对数据的解释程度（值越大，说明回归效果越好）。

- **三分类准确率(ACC_3)**:三分类准确率是指在一个有三个不同类别的分类问题中，模型正确预测的样本比例。通常情况下，三分类准确率可以被定义为模型对所有三个类别中正确预测的样本数目的比例,即：

$$\text{三分类准确率} = \frac{\text{预测正确的样本数}}{\text{总样本数}}$$

这里，“预测正确的样本数”是指模型在所有预测中正确预测的样本数目，总样本数则是所有样本的数量。显然，值越大，也说明回归效果越好。（后简称ACC_3）

6.4.2 构建基准模型：线性回归

线性回归是一种用于描述自变量与因变量之间线性关系的统计学方法。其基本思想是通过拟合一条直线（或超平面）来最好地拟合数据点，从而预测因变量的取值。由于该模型原理较为简单，我们选择将其作为基准模型，并借其性质来进一步探索指标数据与洪水概率的深层次关系。

下面详细介绍线性回归的过程和数学原理：

1. **线性关系建模**: 线性回归假设因变量 y 与自变量 x 之间存在线性关系, 可以用一条直线或超平面表示。数学表达式为

$$y = \omega_0 + \omega_1 x_1 + \omega_2 x_2 + \cdots + \omega_n x_n = \omega^T x$$

其中 $x_1, x_2, \dots, x_n$ 为自变量, $\omega_0, \omega_1, \dots, \omega_n$ 为对应权重, $\omega = (\omega_0, \omega_1, \dots, \omega_n)$ 称之为权重向量, $x = (1, x_1, x_2, \dots, x_n)$ 称之为特征向量。

2. **最小化损失函数**: 通常将均方误差 (Mean Squared Error, MSE) 作为损失函数, 即:

$$L = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 = \frac{1}{m} \sum_{i=1}^m (y_i - \omega^T x_i)^2$$

其中 y_i 为真实值, $\hat{y}_i$ 为预测值。

线性回归的目标是最小化损失函数, 即通过最小化实际值与预测值之间的误差平方和来得到最优参数。采用最小二乘法得到最优参数

$$\omega = (X^T X)^{-1} X^T y$$

其中 X 为一 $m \times (n+1)$ 的矩阵, 称之为特征矩阵 (每一行均为不同样本点的特征向量的矩阵, 故 n 指标数, m 为样本数目), $y = (y_1, y_2, \dots, y_m)$ 是由对应样本的应变量构成的向量。

直接利用所有指标进行线性回归模型训练, 通过该模型得到各个指标特征的权重如下:

表 8 各指标权重 ($\omega_0 = -0.0534$)					
指标	季风强度	地形排水	河流管理	森林砍伐	城市化
权重	0.0056	0.0056	0.0057	0.0057	0.0057
指标	气候变化	大坝质量	淤积	农业实践	侵蚀
权重	0.0057	0.0057	0.0056	0.0056	0.0056
指标	无效防灾	排水系统	海岸脆弱性	滑坡	流域
权重	0.0056	0.0056	0.0057	0.0056	0.0056
指标	基础设施恶化	人口得分	湿地损失	规划不足	政策因素
权重	0.0056	0.0057	0.0056	0.0056	0.0056

可见, 各指标权重差距并不大, 因此需要对模型性能进行进一步评估
线性回归模型得出的基准结果如下:

表 9 线性回归模型 (Org-20)	
R2 分数	ACC_3
0.84444	0.76743

其中 Org-20 表示选用原始 20 指标。

可见, 基本的线性回归模型便可以获得较为优秀的结果, 由此推断指标数据与洪水概率之间的线性相关性是较高的, 至此我们得到了洪水概率预测的基准模型, 接下来我们选取关键指标用以进一步建立预警评价模型。

6.4.3 选取关键指标

在上述线性回归模型可以发现，指标数据未经特殊处理的情况下依旧与洪水概率有着较强的线性相关性，因此，为了构造关键引导变量，将所有指标之和求出，并与其相应洪水概率可视化如下：

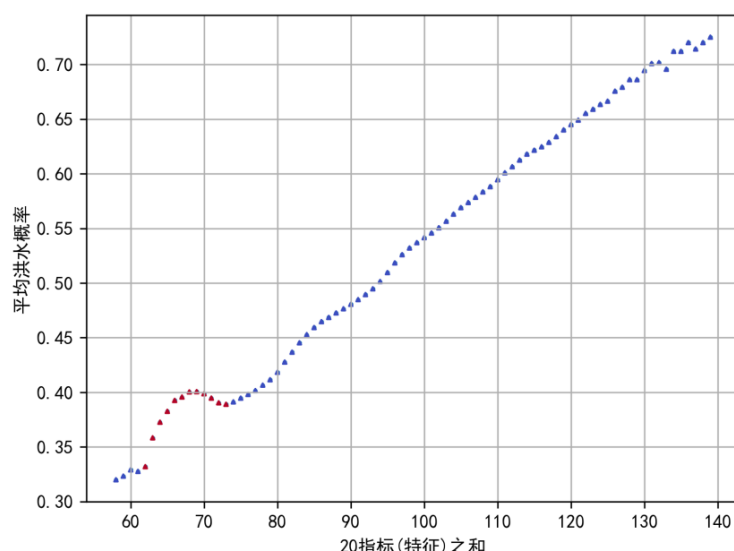


图 8 指标和与平均洪水概率描述

可以观察到，洪水发生概率与指标和基本呈正相关。因此我们选用指标和作为关键指标，引导模型训练。同时，观察到线性关系中存在特殊值（红色部分）。为了保证模型的低灵敏度（强泛化性、强鲁棒性），将特殊值作为数据噪声放入指标考虑范围内。

同时，由于指标维度上数据参差不齐，为了减少在特定维度上的指标数值差异，在样本维度上对所有指标进行降序排序以减少指标维度上的差异性，构造排序指标。

在这一步骤，我们通过对数据的可视化探索选取**指标和**作为一个关键指标，并且将季风强度、地形排水等指标在样本维度上从小到大排序，构造**排序指标**，作为另一个关键指标。

6.4.4 CatBoost 进一步建立模型

为了更好地介绍 CatBoost 之前我们先简单地讲解几个概念：

- **决策树**：决策树是一个预测模型，它代表的是对象属性与对象值之间的一种映射关系(李航 2022: 57-75)。树中每个节点表示某个对象，而每个分叉路径则代表某个可能的属性值，而每个叶节点则对应从根节点到该叶节点所经历的路径所表示的对象的值。如图所示就是一个决策树。

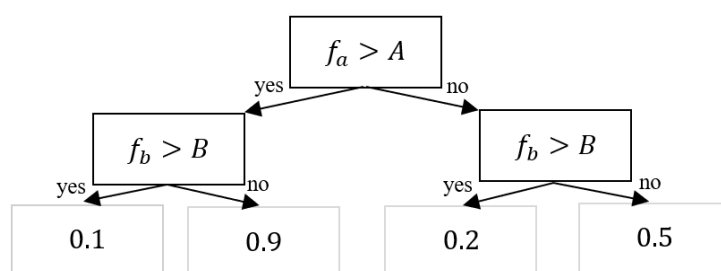


图 9 朴素决策树示意图

决策树的结果可以表示为：

$$\hat{y}(x) = \sum_{m=1}^M C_m I(x \in R_m)$$

其中 $\hat{y}(x)$ 是输入样本 x 的预测值， M 为决策树的叶节点数量， R_m 是决策树第 m 个叶节点的区域， C_m 是第 m 个叶节点的类别输出， I 是指示函数，表示如果 x 属于 R_m 则为 1，否则为 0。

- **集成学习方法：**集成学习方法是一种机器学习策略，旨在通过组合多个学习器的预测结果来改善整体的预测性能(李航 2022: 131-146)。它可以通过结合多个模型的优点，来提高泛化能力和预测准确性，特别是在处理复杂问题或数据集中存在噪声的情况下表现良好。常见的集成学习方法有 Bagging (Bootstrap Aggregating), Boosting, Stacking 等等，后文使用的 CatBoost 就是一种通过串行训练多个弱学习器，每个学习器专注于修正前序学习器的错误，最终构建一个强大的集成模型的集成学习方法。
- **梯度提升树：**梯度提升树(李航 2022: 131-146)是一种强大的集成学习方法，通过结合多棵决策树来提升预测性能。它基于逐步优化损失函数的梯度，每一棵树都专注于拟合前一棵树的残差，最终通过组合多个模型来减少预测误差，广泛应用于回归和分类任务中，特别适合处理复杂的非线性关系和高维数据。

根据以上概念我们可以得到 CatBoost 的基本原理：

CatBoost [1]是一种基于对称决策树(oblivious trees)为基学习器(弱学习器)实现的参数较少、支持类别型变量和高准确性的 GBDT (Gradient Boosting Decision Tree) 框架。其过程如下：

1. **定义模型：**CatBoost 回归模型可以表示为一个集成模型，由多个决策树组成。假设我们有 K 棵树，模型可以表示为：

$$F(x) = \sum_{k=1}^K f_k(x)$$

其中， x 是输入特征向量， $f_k(x)$ 是第 k 棵树的预测函数。

2. **定义损失函数：**为了训练模型，我们需要定义一个损失函数来衡量模型的预测误差。通常在回归任务中，我们使用均方误差 (MSE) 作为损失函数：

$$L = MSE = \frac{1}{N} \sum_{i=1}^N (y_i - F(x_i))^2$$

其中， N 是训练样本的数量， y_i 是第 i 个样本的真实标签。

3. **训练过程：**CatBoost 使用梯度提升算法来逐步优化模型。在每一步中，根据当前模型的残差计算新的决策树 f_k ，使得损失函数最小化。每棵树的建立过程可以表示为：

$$f_k = \arg \min_f \sum_{i=1}^N L(y_i, F_{k-1}(x_i) + f(x_i))$$

这里 $F_{k-1}(x_i)$ 是前 $k-1$ 棵树的累积预测， L 是损失函数。

4. **正则化**: CatBoost 在训练过程中还包括一些正则化技术, 如树的深度限制、学习率控制等, 以防止过拟合并提高泛化能力。
5. **预测**: 训练完成后, 通过将所有树的预测累加来得到最终的预测结果:

$$\hat{y} = F(x) = \sum_{k=1}^K f_k(x)$$

这就是 CatBoost 回归模型的基本数学过程: 通过梯度提升的方式, 逐步构建多棵决策树, 并结合它们的预测来最小化损失函数, 从而得到最优的回归预测模型。

最终, 我们得到的 CatBoost 回归模型效果如下:

表 10 CatBoost 回归模型 (Sorted-20, Sum-1)	
R2 分数	ACC_3
0.86923	0.76290

其中 Sorted-20 代表升序排列的原始 20 指标, Sum-1 代表 1 个指标之和。

可见, CatBoost 在经过严谨分析的特征工程之后在回归任务上优于基准模型, 三分类任务准确度与基准模型在伯仲之间。

6.4.5 K 折交叉灵敏性验证

K 折交叉验证 (K-fold Cross-Validation) 是一种常用的交叉验证技术, 用于评估模型在数据集上的性能和泛化能力。它将数据集分成 K 个子集, 每个子集称为一个折 (fold)。K 折交叉验证的过程如下:

1. **数据集划分**:
 - 将数据集分成 K 个大致相等的部分 (折)。
 - 每个折依次作为验证集, 其余 K-1 个折作为训练集。
2. **交叉验证过程**:
 - 第一轮: 将第一折作为验证集, 其余 K-1 折作为训练集, 训练模型并在第一折上进行评估。
 - 第二轮: 将第二折作为验证集, 其余 K-1 折作为训练集, 训练模型并在第二折上进行评估。
 - 依此类推, 直到第 K 轮。
3. **评估指标**: 每轮验证后得到一个评估指标 (比如准确率、精确率、召回率等)。最终的模型性能评估通常是五轮验证结果的平均值, 这样可以减少因为特定数据分割而引入的偏差。

此处我们采用 **5 折交叉验证**, 得到的结果如下:

表 11 CatBoost 回归模型 5 折交叉验证						
	1 折	2 折	3 折	4 折	5 折	均值
R2 分数	0.87019	0.86971	0.86849	0.86885	0.86892	0.86923
ACC_3	0.76247	0.76296	0.76290	0.76295	0.76324	0.76290

可见, 模型在 5 折交叉验证的训练过程中保持了在回归与分类问题上的稳定且优异的性能, 证明了其**低灵敏度、高泛化性、高鲁棒性**

七、问题 3 模型的建立与求解

7.1 利用精选指标优化模型 (基于问题 1 的分析)

同时，加入指标和作为关键引导指标，组成全新的 11 维特征，训练 CatBoost 回归模型，性能较基准模型有所提升，而相比排序指标+指标和的模型部分伯仲，训练结果如下：

表 12 CatBoost 回归模型 (Selected-10, Sum-1)	
R2 分数	ACC_3
0.86923	0.76290

其中，Selected-10 代表精选高相关性指标 10 个，Sum-1 代表使用 1 个指标和作为关键指标。

可见，CatBoost(Selected-10, Sum-1)模型与 CatBoost(Sorted-20, Sum-1)的性能基本相当，且均在 R2 分数，即回归任务上优于普通线性回归的性能。

7.2 五指标模型优化

为了在仅选用 5 个关键指标作为特征的限制下优化问题 2 中所建立的 CatBoost 回归模型，本文通过借助划分指标来进行主成分分析，将 20 维指标降维至 4 维关键指标，结合指标和组成 5 个关键指标，实现求解得到最优的 CatBoost 回归模型。

7.2.1 主成分分析 (PCA)

● PCA 简介

主成分分析 (principal component analysis) 是一种通过正交变换，将一系列可能线性相关的变量，转换成一组线性不相关的新变量的数学降维方法。而这些不相关变量也称作主成分(李航 2022: 250-269)。主成分分析在简化运算、去除噪音、数据可视化等方面起到了重要作用。

PCA 算法的大致流程如下：

我们将 m 个 n 维数据记作矩阵 $X_{n \times m} = [\bar{x}_1, \bar{x}_2, \dots, \bar{x}_m]$

1. 对数据去中心化：

$$X = X - \frac{1}{m} \sum_{i=1}^m \bar{x}_i$$

2. 计算协方差矩阵 C ：

$$C = \frac{1}{m} X X^T$$

3. 求解 C 的特征值矩阵 (其中特征值按降序排列),取前 k 列，记作矩阵：

$$P = P_{n \times k}$$

4. 将原始数据对进行投影，得到降维后数据，即：

$$Y_{k \times m} = P_{n \times k}^T X_{n \times m}$$

● 应用 PCA 提取关键特征

PCA 降维提取关键特征的步骤如下：

1. **数据选取：**选取所有 20 维指标作为特征维度，洪水概率为目标维度
2. **特征标准化：**PCA 对数据的尺度敏感，不同尺度的数据可能会导致主成分分析的结果不准确，因此对其进行标准化处理
3. **特征降维：**选取降维比例为 20%，将 20 维特征降维至 4 维，作为关键特征
4. **PCA 可视化：**展示主成分 1、2 的分布情况，并根据目标变量对数据点进行

行了颜色编码，以显示不同类别的数据分布。

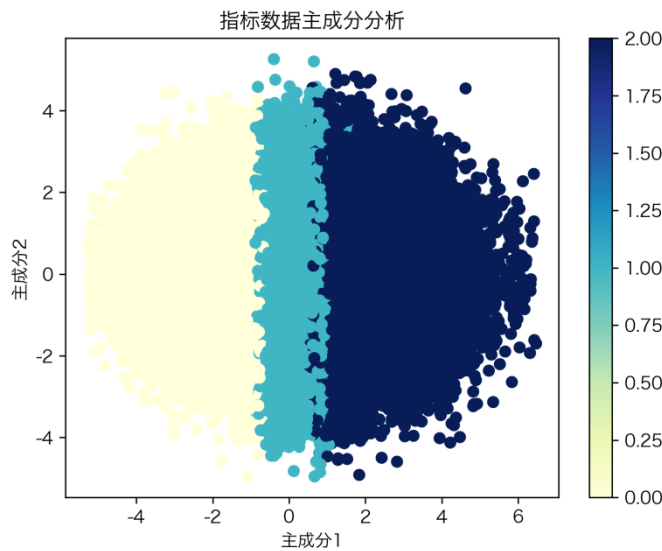


图 10 指标数据主成分分析结果

可见,PCA 数据降维实现了较好的特征分解与关键指标提取工作

7.2.2 最优 CatBoost 回归模型

基于 PCA 降维后的 4 个主成分，辅以指标和组成 5 个关键指标，用以训练 CatBoost 回归模型，并实现了远超先前所有基准、优化模型的效果，实现了 SOTA（State Of The Art）的最优模型：

表 13 SOTA CatBoost 回归模型（PCA-4， Sum-1）	
R2 分数	ACC_3
0.97369	0.90594

其中,PCA-4 代表 PCA 压缩指标到 4 维,Sum-1 代表使用一个指标和作为关键指标,SOTA 即 State Of The Art 其代表最优模型。

可见，PCA 降维处理后的数据结合指标和作为关键指标是极佳的特征选取与融合，实现了精妙的特征工程，使得模型性能大幅提升。

具体模型优化效果及其与基准模型的对比如下表所示：

表 14 洪水概率预测回归模型		
模型	R2 分数	ACC_3
线性回归(Org-20)	0.84444	0.76743
XGBoost(Org-20)	0.80975	0.74846
CatBoost(Org-20)	0.84634	0.76736
CatBoost(Sorted-20,Sum-1)	0.86923	0.76290
CatBoost(Selected-10,Sum-1)	0.86691	0.76281
SOTA CatBoost(PCA-4,Sum-1)	0.97369	0.90594

由此可清晰看到本文对模型优化的进行过程及特征工程带来的极佳优化效果。

八、问题 4 模型的建立与求解

基于问题 2 选取的回归模型 CatBoost，加以问题 3 对指标进行的关键指标选取，实现了最优的回归模型，并将该模型与特征工程应用于 test.csv 数据，并将预

测结果填写进入 submit.csv 中。

按照问题 1 中的数据分析结果,测试集的频数分布直方图与训练集基本相同,而训练集的洪水概率的分布情况基本符合正态分布,因此可以假设测试集的预测洪水概率分布情况也应基本尽量符合正态分布。

对预测结果的概率密度直方图可视化如下:

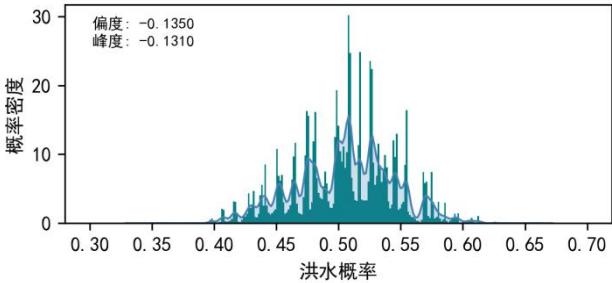


图 11 SOTA CatBoost (PCA-4, Sum-1) 预测洪水发生概率统计直方图

该预测结果分布情况没有明显的偏向性,基本符合正态分布,对比其他性能较弱的两个 CatBoost 回归模型的预测结果概率分布直方图如下:

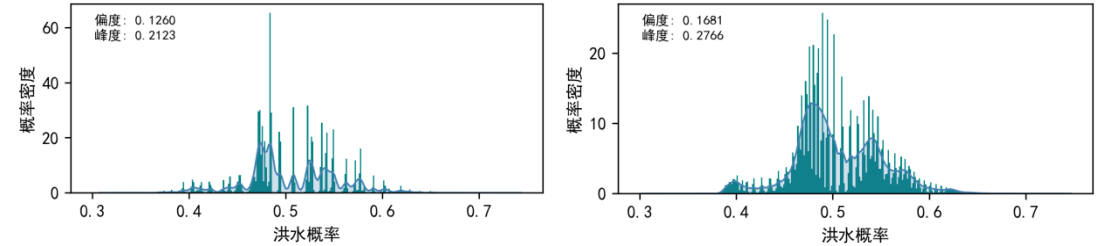


图 12 CatBoost[(Selected-10,Sum-1), (Sorted-20,Sum-1)]预测洪水发生概率统计直方图

性能较弱的两个模型有着明显的预测偏向性,并不符合正态分布,可见,性能最优预测模型的预测结果是较为可信的,这也验证了该模型的鲁棒性。

对最优模型预测结果按正态分布随机采样 1000 条数据,可视化为折线图如下:

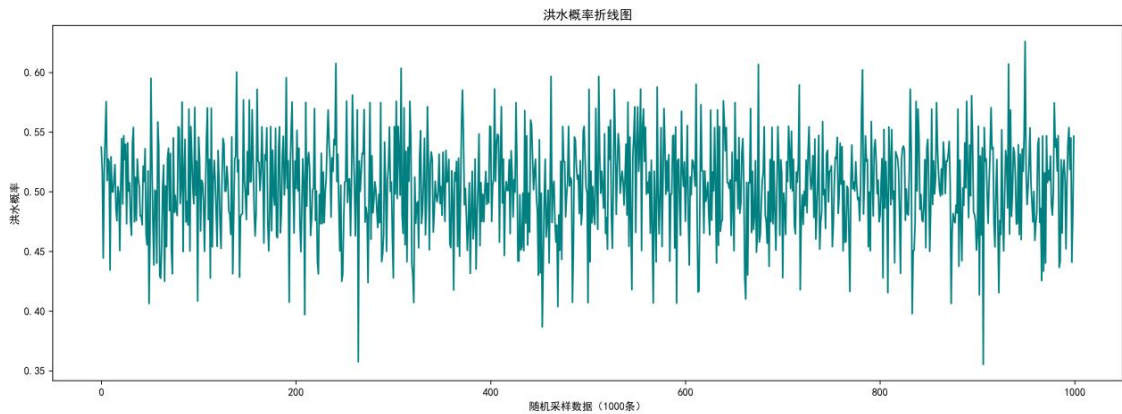


图 13 SOTA CatBoost (PCA-4, Sum-1) 随机采样洪水折线图

可见数据基本在 0.5 周围上下浮动,异常值较少,并不影响整体数据分布情况。

九、模型的评价与推广

9.1 模型评价

(1) 优势

- 1. 量化数据有效：**构建过程引入多种统计量对数据进行定量分析以选取建模方向
- 2. 数据分析严谨：**利用 Spearman 相关系数，量化了各指标与洪水概率之间的相关性，并进行了假设检验，确认了模型特征选择的合理性。
- 3. 特种工程合理有效：**经过对数据逐步地细致分析，最终得出了最优的特征处理方式，实现了性能极佳的模型。
- 4. 性能指标选取合适：**使用了 R^2 分数和三分类准确率（ACC_3）作为模型性能的评估标准，通过这些指标，对不同模型的性能进行了横向比较。
- 5. 验证方法可靠严谨：**采用 K 折交叉验证方法，验证了模型的稳定性和泛化能力，确保模型在未见数据上也能保持良好性能。
- 6. 可视化多样：**通过直方图、折线图等可视化手段，直观展示了模型预测结果的分布情况，进一步分析了预测结果的正态性。

(2) 劣势

- 1. 无法处理时序数据：**由于给定数据的限制，无法在时序上进行深入建模，这会造成模型实用性受到限制，因为现实中很多数据是包含强时序关系的。
- 2. 可识别处理的指标特征太少：**由于给定数据的限制，进行更高特征维度的建模工作，这可能会影响实际情况下模型的学习表征的能力。

9.2 模型推广

基于上述评估结果，我们认为所构建的模型具有较高的实用价值和推广潜力。以下是模型推广的几个方向：

- 1. 数据集扩展：**将模型应用于更广泛地区的洪水灾害数据，以验证模型在不同地理和气候条件下的适用性。
- 2. 实时预测系统：**结合实时气象和水文数据，开发实时洪水灾害预测系统，为防洪减灾提供决策支持。
- 3. 政策制定支持：**模型结果可为政府和相关部门制定防洪策略和应急响应计划提供科学依据。
- 4. 公众教育与预警：**利用模型预测结果，开发公众洪水灾害预警应用程序，提高公众的防灾意识和应对能力。

十、参考文献

- [1] Prokhorenkova L, Gusev G, Vorobev A, et al. CatBoost: unbiased boosting with categorical features[J]. Advances in neural information processing systems, 2018, 31.
- [2] 李航. 《机器学习方法》. 北京: 清华大学出版社, 2022: 57-75, 77-93, 131-146, 216-227, 250-269.

十一、附录

中风险区的各指标分布情况					
	季风强度	地形排水	河流管理	森林砍伐	城市化
平均值	4.884	4.910	4.926	4.915	4.915
最小值	0	0	0	0	0
最大值	16	18	16	17	17
	气候变化	大坝质量	淤积	农业实践	侵蚀
平均值	4.899	4.925	4.900	4.914	4.933
最小值	0	0	0	0	0
最大值	16	16	16	16	18
	无效防灾	排水系统	海岸脆弱性	滑坡	流域
平均值	4.932	4.916	4.924	4.907	4.912
最小值	0	0	0	0	0
最大值	16	17	17	16	16
	基础设施恶化	人口得分	湿地损失	规划不足	政策因素
平均值	4.889	4.905	4.923	4.922	4.911
最小值	0	0	0	0	0
最大值	17	18	17	16	16

高风险区的各指标分布情况					
	季风强度	地形排水	河流管理	森林砍伐	城市化
平均值	5.438	5.423	5.458	5.434	5.430
最小值	0	0	0	0	0
最大值	16	18	16	17	17
	气候变化	大坝质量	淤积	农业实践	侵蚀
平均值	5.434	5.464	5.426	5.432	5.421
最小值	0	0	0	0	0
最大值	17	16	16	16	17
	无效防灾	排水系统	海岸脆弱性	滑坡	流域
平均值	5.424	5.432	5.440	5.424	5.411
最小值	0	0	0	0	0
最大值	16	17	17	16	16
	基础设施恶化	人口得分	湿地损失	规划不足	政策因素
平均值	5.440	5.421	5.440	5.417	5.430
最小值	0	0	0	0	0
最大值	17	17	18	16	16

train.head()

i d	季 风 强 度	地 形 排 水	河 流 管 理	森 林 砍 伐	城 市 化	气 候 变 化	大 坝 质 量	淤 积	农 业 实 践	侵 蚀	.	排 水 系 统	海 岸 脆 弱 性	滑 坡	流 域	基 础 设 施 恶 化	人 口 得 分	湿 地 损 失	规 划 不 足	政 策 因 素	洪 水 概 率
0	5	8	5	8	6	4	4	3	3	4	.	5	3	3	5	4	7	5	7	3	0.4 45
1	6	7	4	4	8	8	3	5	4	6	.	7	2	0	3	5	3	3	4	3	0.4 50
2	6	5	6	7	3	7	1	5	4	5	.	7	3	7	5	6	8	2	3	3	0.5 30
3	3	4	6	5	4	8	4	7	6	8	.	2	4	7	4	4	6	5	7	5	0.5 35
4	5	3	2	6	4	4	3	3	3	3	.	2	2	6	6	4	1	2	3	5	0.4 15

5 rows × 21 columns

train.describe().洪水概率

count 1.048575e+06

mean 5.044697e-01

std 5.103092e-02

min 2.850000e-01

25% 4.700000e-01

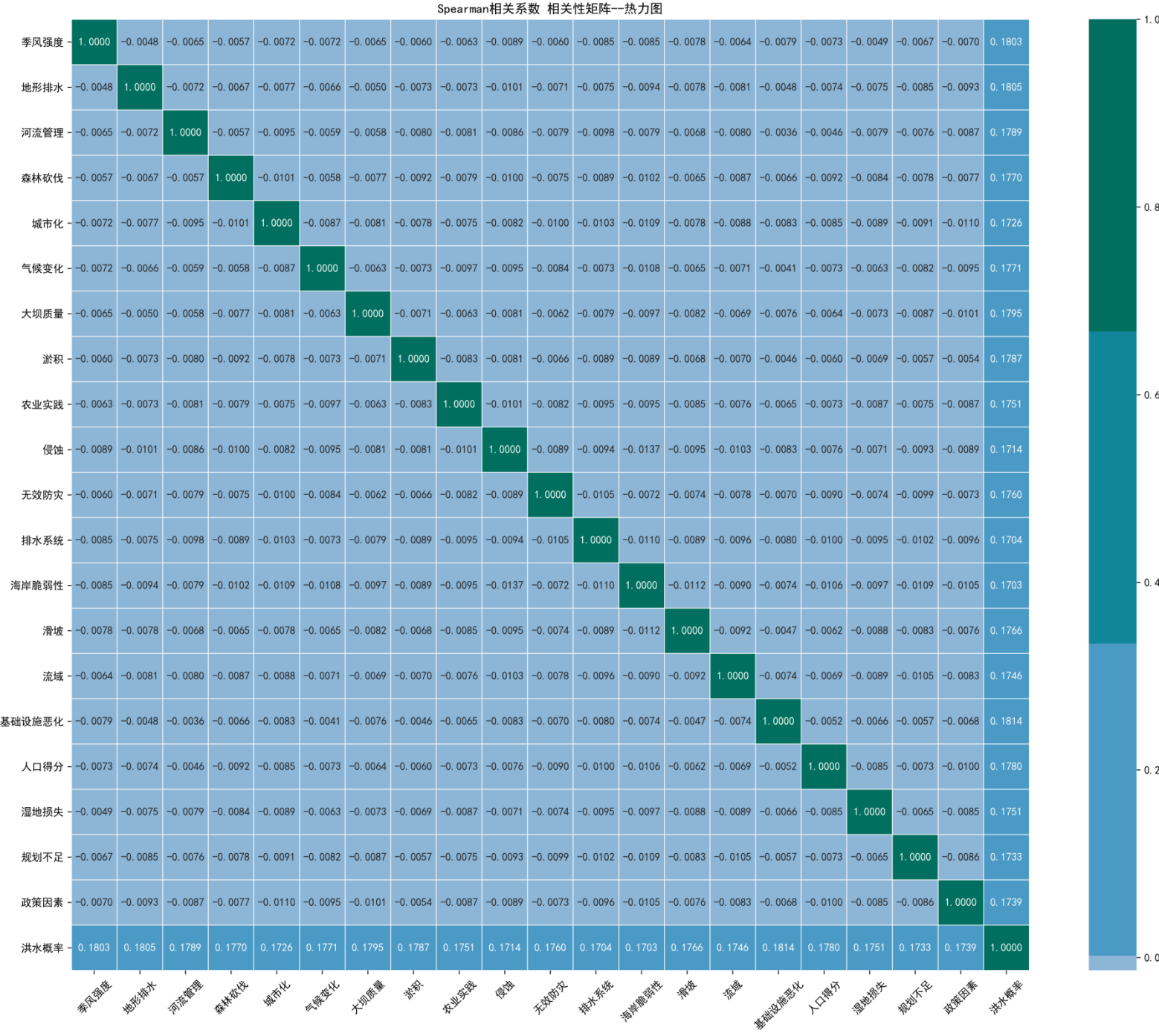
50% 5.050000e-01

75% 5.400000e-01

max 7.250000e-01

Name: 洪水概率, dtype: float64

spearman_corr_matrix

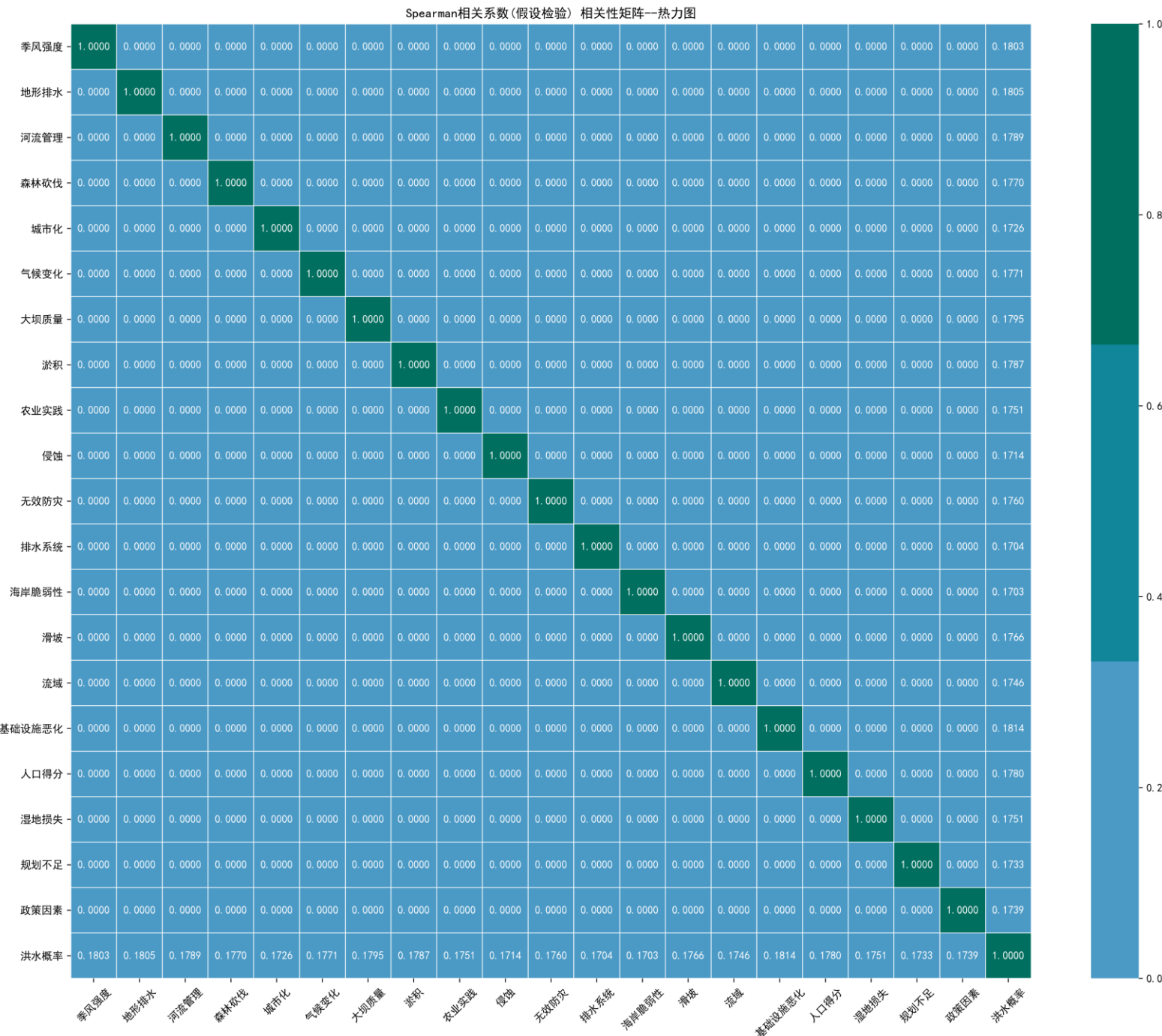


21 rows × 21 columns

flood_corr	
基础设施恶化	0.181399
地形排水	0.180483
季风强度	0.180277
大坝质量	0.179458
河流管理	0.178931
淤积	0.178738
人口得分	0.177993
气候变化	0.177119
森林砍伐	0.177014
滑坡	0.176636
无效防灾	0.175972
农业实践	0.175144
湿地损失	0.175139
流域	0.174641
政策因素	0.173873
规划不足	0.173254
城市化	0.172649
侵蚀	0.171430
排水系统	0.170441
海岸脆弱性	0.170289

Name: 洪水概率, dtype: float64

corrected_df



21 rows × 21 columns

PCA

[0.08241561 0.04858298 0.04843683 0.04841333]

[0.05102115 0.05090417 0.05083524 0.05082428]

Submit

	id	洪水概率	Unnamed: 2	Unnamed: 3
0	1117957	0.574503	NaN	NaN
1	1117958	0.554082	NaN	
2	1117959	0.538789	NaN	NaN
3	1117960	0.526443	NaN	NaN
4	1117961	0.476134	NaN	NaN
5	1117962	0.508957	NaN	NaN
6	1117963	0.519356	NaN	NaN
7	1117964	0.508387	NaN	NaN
8	1117965	0.527218	NaN	NaN
9	1117966	0.428156	NaN	NaN

OLS Regression Results

Dep. Variable: 洪水概率 R-squared: 0.845
 Model: OLS Adj. R-squared: 0.845
 Method: Least Squares F-statistic: 2.859e+05
 Date: Sun, 07 Jul 2024 Prob (F-statistic): 0.00
 Time: 17:10:41 Log-Likelihood: 2.6095e+06
 No. Observations: 1048575 AIC: -5.219e+06
 Df Residuals: 1048554 BIC: -5.219e+06
 Df Model: 20

Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	-0.0534	0.000	-227.919	0.000	-0.054	-0.053
季风强度	0.0056	9.55e-06	587.709	0.000	0.006	0.006
地形排水	0.0056	9.38e-06	601.465	0.000	0.006	0.006
河流管理	0.0057	9.48e-06	597.629	0.000	0.006	0.006
森林砍伐	0.0057	9.57e-06	593.667	0.000	0.006	0.006

城市化	0.0057	9.43e-06	599.498	0.000	0.006	0.006
气候变化	0.0057	9.54e-06	593.585	0.000	0.006	0.006
大坝质量	0.0057	9.43e-06	599.400	0.000	0.006	0.006
淤积	0.0056	9.51e-06	593.297	0.000	0.006	0.006
农业实践	0.0056	9.5e-06	593.369	0.000	0.006	0.006
侵蚀	0.0056	9.43e-06	598.734	0.000	0.006	0.006
无效防灾	0.0056	9.45e-06	596.449	0.000	0.006	0.006
排水系统	0.0056	9.48e-06	594.350	0.000	0.006	0.006
海岸脆弱性	0.0057	9.41e-06	603.088	0.000	0.006	0.006
滑坡	0.0056	9.45e-06	596.738	0.000	0.006	0.006
流域	0.0056	9.44e-06	598.644	0.000	0.006	0.006
基础设施恶化	0.0056	9.51e-06	590.926	0.000	0.006	0.006
人口得分	0.0057	9.47e-06	599.639	0.000	0.006	0.006
湿地损失	0.0056	9.5e-06	593.667	0.000	0.006	0.006
规划不足	0.0056	9.44e-06	594.258	0.000	0.006	0.006
政策因素	0.0056	9.4e-06	600.974	0.000	0.006	0.006
Omnibus:	94200.120	Durbin-Watson:	2.000			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	139790.413			
Skew:	0.704	Prob(JB):	0.00			
Kurtosis:	4.102	Cond. No.	265.			

代码

问题 1、3、4

```

1. # %%
2. import numpy as np
3. import pandas as pd
4. import seaborn as sns
5. import matplotlib as mpl
6. import scipy.stats as stats
7. import matplotlib.pyplot as plt
8.
9. from os.path import join
10. from statsmodels.stats.multitest import multipletests
11.
12. import warnings
13. warnings.filterwarnings("ignore")
14.

```

```

15. # %%
16. plt.rcParams['font.sans-serif'] = ['SimHei']
17. plt.rcParams['axes.unicode_minus'] = False
18.
19. # %% [markdown]
20. # # Problem 1
21. #
22. # 问题 1. 请分析附件 train.csv 中的数据，分析并可视化上述 20 个指标中，哪些
    指标与洪水的发生有着密切的关联？哪些指标与洪水发生的相关性不大？并分析可能的
    原因，然后针对洪水的提前预防，提出你们合理的建议和措施。
23.
24. # %%
25. data_root = r'/home/drew/Desktop/MathModeling/APMCM24_cn/data'
26.
27. train_path = join(data_root, 'train.csv')
28. test_path = join(data_root, 'test.csv')
29.
30. train = pd.read_csv(train_path, index_col='id')
31. test = pd.read_csv(test_path, index_col='id')
32.
33. initial_features = list(test.columns)[: -1]
34.
35. train.head()
36.
37. # %%
38. train.isna().any()
39.
40. # %%
41. test.isna().any()
42.
43. # %%
44. print(initial_features)
45.
46. # %% [markdown]
47. # ## 洪水发生概率分布情况
48.
49. # %% [markdown]
50. # 由于测试集中的洪水发生概率仍旧是缺失值，因此无法直接计算测试集的洪水概率分
    布情况，此处仅对训练集进行统计
51.
52. # %%
53. train.describe().洪水概率
54.
55. # %% [markdown]

```

```

56. # 对洪水发生概率进行统计，得出其最小值为 0.285，最大值为 0.725
57. #
58. # 因此对其进行可视化时扩展设置信度为 0.005，则扩展下界到 0.2825，扩展上界到
    0.7275，构造 90 个箱体进行概率密度的可视化
59.
60. # %%
61. np.linspace(0.2825, 0.7275, 90)[:5], (0.7275 - 0.2825)/90
62.
63. # %%
64. flood_df = train.洪水概率
65. # 计算偏度和峰度
66. flood_df_skew = stats.skew(flood_df.dropna()) # 删除 NaN 值并计算偏
    度
67. flood_df_kurt = stats.kurtosis(flood_df.dropna()) # 删除 NaN 值并计算
    峰度
68. plt.figure(figsize=(5, 2), dpi=300)
69. plt.hist(
70.     flood_df,
71.     bins=np.linspace(0.2825, 0.7275, 90),
72.     density=True,
73.     color='teal',
74. )
75. sns.kdeplot(
76.     flood_df,
77.     color='steelblue',
78.     shade=True,
79. )
80. # 添加偏度和峰度到图的左上角
81. plt.text(
82.     0.05, # x 位置
83.     0.95, # y 位置，可以根据你的图形布局进行调整
84.     f'偏度: {flood_df_skew:.4f}\n峰度: {flood_df_kurt:.4f}', # 要显示
        的文本
85.     horizontalalignment='left', # 水平对齐方式
86.     verticalalignment='top', # 垂直对齐方式
87.     transform=plt.gca().transAxes, # 将位置指定为图形坐标轴的比例
88.     fontsize=8
89. )
90. plt.ylabel('概率密度')
91. plt.xlabel('洪水概率')
92. plt.show()
93.
94. # %%
95. print(f'偏度: {flood_df_skew:.4f}\n峰度: {flood_df_kurt:.4f}')

```

```

96.
97. # %% [markdown]
98. # **洪水分布情况**：
99. #     - 偏度（Skewness）：0.0481，接近 0，表明洪水发生概率的分布接近对称，没有明显的右偏或左偏。
100. #     - 峰度（Kurtosis）：-0.0390，表明洪水发生概率的分布比正态分布稍微平坦一些，也就是说，其峰值不是很明显，尾部相对更厚。
101.
102. # %% [markdown]
103. # ## 特征分布情况
104. #
105. # 对每个特征进行统计，并统计出训练集和测试集中每个特征的分布情况
106.
107. # %%
108. ncol = 4
109. nrow = int(np.ceil(len(initial_features) / 4))
110. plt.figure(figsize=(ncol*3, nrow*2.5), dpi=300)
111. pal_train = sns.color_palette("PuBuGn")
112. pal_test = sns.cubehelix_palette(11)
113. for i,col in enumerate(initial_features):
114.     ax = plt.subplot(nrow, ncol, i+1)
115.     sns.countplot(train,x=col,ax=ax,palette=pal_train)
116.     sns.countplot(test,x=col,ax=ax,palette=pal_test)
117.     ax.set_title(col)
118.     ax.xaxis.set_major_locator(mpl.ticker.MaxNLocator(integer=True
    ))
119.     ax.set_xlabel("频数")
120. plt.tight_layout()
121.
122. # %%
123. # 假设 initial_features 是一个包含至少两个特征名称的列表
124. # 假设 train 和 test 是包含这些特征的 DataFrame
125. from matplotlib import ticker
126.
127. ncol = 4
128. nrow = int(np.ceil(len(initial_features) / 4)) # 计算需要的行数
129.
130. plt.figure(figsize=(ncol*3, nrow*2.5), dpi=300)
131.
132. # 创建颜色方案
133. pal_train = sns.color_palette("PuBuGn")
134. pal_test = sns.cubehelix_palette(11)
135.
136. # 仅绘制前两个特征

```



```

137. for i, col in enumerate(initial_features[:2]): # 修改这里以仅迭代前
    两个特征
138.     ax = plt.subplot(nrow, ncol, i + 1) # 子图布局
139.     sns.countplot(train[train[col].notnull()], x=col, ax=ax, palette=
        pal_train) # 绘制训练集的计数图
140.     sns.countplot(test[test[col].notnull()], x=col, ax=ax, palette
        =pal_test) # 绘制测试集的计数图
141.     ax.set_title(col) # 设置子图标题
142.     ax.xaxis.set_major_locator(ticker.MaxNLocator(integer=True))
        # 设置 x 轴刻度为整数
143.     ax.set_xlabel("频数")
144.
145. plt.tight_layout() # 调整子图布局以避免重叠
146. plt.show() # 显示图形
147.
148. # %%
149. stats_results = {'特征': [], '训练集偏度系数': [], '训练集峰度系数
        ': [],
150.                  '测试集偏度系数': [], '测试集峰度系数': []}
151.
152. for col in initial_features:
153.     train_skew = stats.skew(train[col].dropna())
154.     train_kurt = stats.kurtosis(train[col].dropna())
155.     test_skew = stats.skew(test[col].dropna())
156.     test_kurt = stats.kurtosis(test[col].dropna())
157.
158.     stats_results['特征'].append(col)
159.     stats_results['训练集偏度系数'].append(train_skew)
160.     stats_results['训练集峰度系数'].append(train_kurt)
161.     stats_results['测试集偏度系数'].append(test_skew)
162.     stats_results['测试集峰度系数'].append(test_kurt)
163.
164. stats_df = pd.DataFrame(stats_results, index=stats_results['特征
        '])
165. stats_df.drop(['特征'], axis=1, inplace=True)
166. stats_df
167.
168. # %% [markdown]
169. # 根据数据，我们可以对洪水发生概率的特征分布情况做出以下分析：
170. #
171. # **特征分布情况**：
172. #     - 所有特征的偏度系数（Skewness）都在 0.2 到 0.5 之间，这表明大多数特
        征都呈现轻微到中等程度的右偏分布。右偏意味着大多数数据点集中在较低的值，而有一些
        极端值在较高的一端。

```

```

173. # - 特征的峰度系数（Kurtosis）大多数在-0.2 到 0.4 之间，这表明大多数特
    征的分布比正态分布更平坦或稍微尖锐，但没有极端的峰度值。
174. #
175. # **训练集与测试集的一致性**：
176. # - 从偏度和峰度系数来看，训练集和测试集的特征分布大致相似，这表明模型
    在训练和测试时使用的是统计特性相似的数据。
177. #
178. # **数据的正态性**：
179. # - 由于大多数特征都表现出偏度，这意味着它们不是正态分布的。峰度系数也
    表明这些特征不遵循正态分布的典型形状。
180. # - 这可能对某些需要数据正态分布假设的统计测试或模型有影响，例如线性回
    归或 t 检验。
181. #
182. # **数据预处理**：
183. # - 根据特征的分布情况，可能需要进行一些数据预处理，如对数转换或
    Box-Cox 转换，以减少偏度，使数据更接近正态分布，特别是如果选择的模型假设数据
    正态性。
184. #
185.
186. # %% [markdown]
187. # ## 特征相关性分析
188. #
189. # 对特征相关性进行分析，并观察特征之间的相关性以及特征与洪水概率之间的相关
    性
190.
191. # %%
192. corr_features = initial_features + ['洪水概率']
193. cc = np.corrcoef(train[corr_features], rowvar=False)
194. plt.figure(figsize=(20, 16), dpi=300)
195. sns.heatmap(cc, center=0, cmap=pal_train, annot=True, fmt='.4f',
196.              xticklabels=corr_features, yticklabels=corr_features,
197.              linewidths=0.3, linecolor='white')
198. plt.xticks(rotation=45)
199. plt.title('Pearson 相关系数 相关性矩阵--热力图')
200. plt.show()
201.
202. # %%
203. spearman_corr_matrix = train[corr_features].corr(
204.     method='spearman', min_periods=1, numeric_only=True)
205.
206. plt.figure(figsize=(20, 16), dpi=300)
207. sns.heatmap(spearman_corr_matrix, center=0, cmap=pal_train, annot=
    True, fmt='.4f',

```

```

208.             xticklabels=corr_features, yticklabels=corr_features,
209.             linewidths=0.3, linecolor='white')
210. plt.title('Spearman 相关系数 相关性矩阵--热力图')
211. plt.xticks(rotation=45)
212. plt.show()
213.
214. # %%
215. spearman_corr_matrix
216.
217. # %%
218. flood_corr = spearman_corr_matrix.洪水概率.drop(['洪水概率'])
219. flood_corr.sort_values(inplace=True, ascending=False)
220. flood_corr
221.
222. # %% [markdown]
223. # **相关性分析**:
224. #     - 已经计算了特征之间的斯皮尔曼相关系数，这将有助于识别特征间的单调关
      系，以及可能存在的多重共线性问题。
225. #
226. # **多重比较校正**:
227. #     - 进行假设检验，需要考虑使用多重比较校正方法，如 Bonferroni 校正，以
      控制第一类错误率。
228.
229. # %%
230. # 初始化一个 DataFrame 来存储 p 值
231. p_values = np.zeros((len(corr_features), len(corr_features)))
232. corrected_p_values = np.zeros((len(corr_features), len(corr_features)))
233.
234. alpha = 0.005
235. # 对相关系数矩阵中的每个系数进行假设检验
236. ans_list = []
237. for i, row in enumerate(spearman_corr_matrix.iterrows()):
238.     temp = []
239.     for j, index in enumerate(corr_features):
240.         p_value = spearman_corr_matrix.loc[row[0], index]
241.         elem = multipletests([p_value], alpha=alpha, method='bonferroni')
242.         not_accept, p_corrected = elem[0], elem[1]
243.         if not_accept:
244.             corrected_p_values[i, j] = 0
245.         else:
246.             corrected_p_values[i, j] = p_corrected

```

```

247.
248.         temp.append(elem)
249.         ans_list.append(temp)
250.
251. # 展示假设检验形式
252. # for row in ans_list:
253. #     print(row)
254.
255. corrected_df = pd.DataFrame(corrected_p_values, columns=corr_features,
                               index=corr_features)
256. corrected_df
257.
258. # %%
259. plt.figure(figsize=(20, 16), dpi=300)
260. sns.heatmap(corrected_df, center=0, cmap=pal_train, annot=True,
               fmt='.4f',
261.             xticklabels=corr_features, yticklabels=corr_features,
262.             linewidths=0.3, linecolor='white')
263. plt.title('Spearman 相关系数(假设检验) 相关性矩阵--热力图')
264. plt.xticks(rotation=45)
265. plt.show()
266.
267. # %%
268. corrected_flood = corrected_df['洪水概率'].drop(['洪水概率']).sort_values(ascending=False)
269. corrected_flood, corrected_flood.index
270.
271. # %%
272. corrected_flood.describe()
273.
274. # %%
275. train = pd.read_csv(join(data_root, 'train_cluster.csv'), index_col=0)
276. test = pd.read_csv(join(data_root, 'test.csv'), index_col=0)
277.
278. # %%
279. import numpy as np
280. import pandas as pd
281. from sklearn.decomposition import PCA
282. from sklearn.preprocessing import StandardScaler
283.
284. def pca(data):
285.     X = data.iloc[:, :-1] # 特征数据

```

```

286.     y = data.iloc[:, -1] # 目标变量
287.     scaler = StandardScaler()
288.     X_scaled = scaler.fit_transform(X)
289.     _pca = PCA(n_components=0.20)
290.     _pca.fit(X_scaled)
291.     X_pca = _pca.transform(X_scaled)
292.     print(_pca.explained_variance_ratio_)
293.     return X_pca, y
294.
295. train_pca, y_train = pca(train)
296. test_pca, y_test = pca(test)
297.
298. # %%
299. dims = train_pca.shape[-1]
300. pca_index = [f'PCA_{i+1}' for i in range(dims)]
301. train[pca_index] = train_pca
302. test[pca_index] = test_pca
303. train['SUM'] = train[initial_features].sum(axis=1)
304. test['SUM'] = test[initial_features].sum(axis=1)
305. train.head()
306.
307. # %%
308. test.head()
309.
310. # %%
311. chosen_index = list(corrected_flood.index[:10]) + ['SUM']
312. chosen_index, len(chosen_index)
313.
314. # %%
315. plt.figure(dpi=300)
316. plt.scatter(train_pca[:, 0], train_pca[:, 1], c=y_train)
317. plt.colorbar()
318. plt.xlabel('主成分 1')
319. plt.ylabel('主成分 2')
320. plt.title('指标数据主成分分析可视化')
321. plt.show()
322.
323. # %% [markdown]
324. # 训练模型
325.
326. # %%
327. import pandas as pd
328. import numpy as np
329. import matplotlib.pyplot as plt

```

```

330. from matplotlib.ticker import MaxNLocator
331. import seaborn as sns
332. from colorama import Fore, Style
333. import xgboost
334. import lightgbm
335. import catboost
336. import os
337. import datetime
338. import pickle
339. import warnings
340. import umap
341. import statsmodels.api as sm
342.
343. from sklearn.base import clone
344. from sklearn.model_selection import KFold, StratifiedKFold
345. from sklearn.pipeline import make_pipeline, Pipeline
346. from sklearn.decomposition import PCA
347. from sklearn.preprocessing import StandardScaler, PolynomialFeatures, SplineTransformer, OneHotEncoder
348. from sklearn.kernel_approximation import Nystroem
349. from sklearn.linear_model import Ridge, LinearRegression
350. from sklearn.ensemble import ExtraTreesRegressor, RandomForestRegressor
351. from sklearn.metrics import r2_score
352. from sklearn.manifold import TSNE
353. from sklearn.compose import TransformedTargetRegressor
354.
355. low, mid, high = (
356.     [0, 0.4724670441310068],
357.     [0.4724670483748616, 0.5375587160676034],
358.     [0.5375587203114581, 1]
359. )
360.
361. # %%
362. # Configuration
363. # Produce a submission file (you can set this to false if you only
364. # want to see the cross-validation results)
365. COMPUTE_TEST_PRED = True
366.
367. # Containers for results
368. oof, test_pred = {}, {}
369.
370. kf = KFold(n_splits=5, shuffle=True, random_state=1)

```

```

371.
372. SINGLE_FOLD = False
373.
374. weight_dir = r'/home/drew/Desktop/MathModeling/APMCM24_cn/weights'
375.
376. from joblib import dump
377.
378. def _classify(O_reg, label_cls, low, mid, high):
379.     cnt = 0
380.     for _, (o_reg, l_cls) in enumerate(zip(O_reg, label_cls)):
381.         o_cls = None
382.         if o_reg < low[1]:
383.             o_cls = 0
384.         elif o_reg >= mid[0] and o_reg < mid[1]:
385.             o_cls = 1
386.         elif o_reg >= high[0] and o_reg <= high[1]:
387.             o_cls = 2
388.         if o_cls == l_cls:
389.             cnt += 1
390.     return cnt
391.
392. def cross_validate(model, label, features=initial_features, n_repe
    ats=1):
393.     """Compute out-of-fold and test predictions for a given model.
394.
395.     Out-of-fold and test predictions are stored in the global vari
    ables
396.     oof and test_pred, respectively.
397.
398.     If n_repeats > 1, the model is trained several times with diff
    erent seeds.
399.     """
400.     best_model = None
401.     best_score = float('-inf')
402.
403.     start_time = datetime.datetime.now()
404.     scores = []
405.     oof_preds = np.full_like(train.洪水概率, np.nan, dtype=float)
406.     cls_cnt, sample_num = 0, 0
407.     for fold, (idx_tr, idx_va) in enumerate(kf.split(train)):
408.         X_train = train.iloc[idx_tr][features]
409.         X_val = train.iloc[idx_va][features]

```

```

410.         y_train = train.iloc[idx_tr].洪水概率
411.         y_val = train.iloc[idx_va].洪水概率
412.         y_label = train.iloc[idx_va].聚类
413.
414.         y_pred = np.zeros_like(y_val, dtype=float)
415.         for i in range(n_repeats):
416.             m = clone(model)
417.             if n_repeats > 1:
418.                 mm = m
419.                 if isinstance(mm, Pipeline):
420.                     mm = mm[-1]
421.                     mm.set_params(random_state=i)
422.                     m.fit(X_train, y_train)
423.                     dump(m, os.path.join(weight_dir, f'{label}_{fold}_{i}.
joblib'))
424.                     y_pred += m.predict(X_val)
425.         y_pred /= n_repeats
426.         sample_num += len(y_label)
427.
428.         cls_cnt += _classify(y_pred, y_label, low, mid, high)
429.         acc_3 = cls_cnt / sample_num
430.         score = r2_score(y_val, y_pred)
431.         print(f"# Fold {fold}: R2={score:.5f}, acc_3={acc_3:.5f}")
432.
433.         if score > best_score:
434.             best_score = score
435.             best_model = clone(m) # 克隆当前模型实例
436.
437.         scores.append(score)
438.         oof_preds[idx_va] = y_pred
439.         if SINGLE_FOLD: break
440.
441.         elapsed_time = datetime.datetime.now() - start_time
442.         print(f"{Fore.GREEN}# Overall: {np.array(scores).mean():.5f} {
label}")
443.         f"{' single fold' if SINGLE_FOLD else ''}"
444.         f" {int(np.round(elapsed_time.total_seconds() / 60))}
min{Style.RESET_ALL}")
445.         oof[label] = oof_preds
446.
447.         if COMPUTE_TEST_PRED:
448.             # Retrain n_repeats times with the whole dataset and avera
ge

```



```

449.         y_pred = np.zeros(len(test), dtype=float)
450.         X_train = train[features]
451.         y_train = train.洪水概率
452.         for i in range(n_repeats):
453.             m = clone(model)
454.             if n_repeats > 1:
455.                 mm = m
456.                 if isinstance(mm, Pipeline):
457.                     mm = mm[-1]
458.                 if isinstance(mm, TransformedTargetRegressor):
459.                     mm = mm.regressor
460.                     mm.set_params(random_state=i)
461.             m.fit(X_train, y_train)
462.             y_pred += m.predict(test[features])
463.         y_pred /= n_repeats
464.         test_pred[label] = y_pred
465.
466.     return best_model
467.
468.     # %% [markdown]
469.     # 11 个特征
470.
471.     # %%
472.     chosen_model = catboost.CatBoostRegressor(
473.         verbose=False,
474.     )
475.     model_1 = cross_validate(
476.         chosen_model,
477.         'CatBoost_chosen',
478.         features=chosen_index,
479.         n_repeats=4
480.     )
481.
482.     # %% [markdown]
483.     # 5 个关键特征
484.
485.     # %%
486.     pca_model = catboost.CatBoostRegressor(
487.         verbose=False,
488.     )
489.     model_2 = cross_validate(
490.         pca_model,
491.         'CatBoost_pca',
492.         features=pca_index,

```

```

493.         n_repeats=4
494.     )
495.
496.     # %%
497.     test_pred
498.
499.     # %%
500.     result_df = pd.read_csv(join(data_root, 'submit.csv'))
501.     if COMPUTE_TEST_PRED:
502.         result_df['洪水概率'] = test_pred['CatBoost_pca']
503.     result_df.head(10)
504.
505.     # %%
506.     result_df.describe()
507.
508.     # %%
509.     flood_df = result_df.洪水概率
510.     # 计算偏度和峰度
511.     flood_df_skew = stats.skew(flood_df.dropna()) # 删除 NaN 值并计算偏
        度
512.     flood_df_kurt = stats.kurtosis(flood_df.dropna()) # 删除 NaN 值并
        计算峰度
513.     plt.figure(figsize=(5, 2), dpi=300)
514.     plt.hist(
515.         flood_df,
516.         bins=np.linspace(0.3,0.7,300),
517.         density=True,
518.         color='teal',
519.     )
520.     sns.kdeplot(
521.         flood_df,
522.         color='steelblue',
523.         shade=True,
524.     )
525.     # 添加偏度和峰度到图的左上角
526.     plt.text(
527.         0.05, # x 位置
528.         0.95, # y 位置, 可以根据你的图形布局进行调整
529.         f'偏度: {flood_df_skew:.4f}\n峰度: {flood_df_kurt:.4f}', # 要
        显示的文本
530.         horizontalalignment='left', # 水平对齐方式
531.         verticalalignment='top', # 垂直对齐方式
532.         transform=plt.gca().transAxes, # 将位置指定为图形坐标轴的比例
533.         fontsize=8

```

```

534. )
535. plt.ylabel('概率密度')
536. plt.xlabel('洪水概率')
537. plt.show()
538.
539. # %%
540. flood_probabilities = result_df['洪水概率
    '].sample(1000).dropna() # 删除 NaN 值
541.
542. plt.figure(figsize=(18, 6),dpi=300)
543.
544. plt.plot(
545.     flood_probabilities.values,
546.     label='洪水概率',
547.     color='teal'
548. )
549. plt.title('洪水概率折线图')
550. plt.ylabel('洪水概率')
551. plt.xlabel('随机采样数据 (1000 条)')
552. plt.show()
553.
554. # %%
555. result_df.to_csv('submit.csv', index=False)
556.
557. # %%
558. result_df_nxt = result_df.copy()
559.
560. if COMPUTE_TEST_PRED:
561.     result_df_nxt['洪水概率'] = test_pred['CatBoost_chosen']
562. result_df_nxt.head(10)
563.
564. flood_df = result_df_nxt.洪水概率
565. # 计算偏度和峰度
566. flood_df_skew = stats.skew(flood_df.dropna()) # 删除 NaN 值并计算偏
    度
567. flood_df_kurt = stats.kurtosis(flood_df.dropna()) # 删除 NaN 值并
    计算峰度
568. plt.figure(figsize=(5, 2), dpi=300)
569. plt.hist(
570.     flood_df,
571.     bins=np.linspace(0.3,0.7,300),
572.     density=True,
573.     color='teal',
574. )

```

```

575. sns.kdeplot(
576.     flood_df,
577.     color='steelblue',
578.     shade=True,
579. )
580. # 添加偏度和峰度到图的左上角
581. plt.text(
582.     0.05, # x 位置
583.     0.95, # y 位置, 可以根据你的图形布局进行调整
584.     f'偏度: {flood_df_skew:.4f}\n峰度: {flood_df_kurt:.4f}', # 要
        显示的文本
585.     horizontalalignment='left', # 水平对齐方式
586.     verticalalignment='top', # 垂直对齐方式
587.     transform=plt.gca().transAxes, # 将位置指定为图形坐标轴的比例
588.     fontsize=8
589. )
590. plt.ylabel('概率密度')
591. plt.xlabel('洪水概率')
592. plt.show()

```

问题 2

```

1. # %%
2. import numpy as np
3. import pandas as pd
4. import seaborn as sns
5. import sklearn as skl
6. import matplotlib as mpl
7. import scipy.stats as stats
8. import matplotlib.pyplot as plt
9.
10. from os.path import join
11. from sklearn.cluster import KMeans, AgglomerativeClustering
12. from sklearn.preprocessing import MinMaxScaler
13. from sklearn.linear_model import LogisticRegression
14. from sklearn.model_selection import train_test_split
15.
16. import warnings
17. warnings.filterwarnings("ignore")
18.
19. plt.rcParams['font.sans-serif'] = ['SimHei']
20. plt.rcParams['axes.unicode_minus'] = False
21.
22. pal_train = sns.color_palette("PuBuGn")
23. pal_train_rev = pal_train.as_hex()[::-1]
24. pal_train_darker = sns.color_palette("PuBuGn_d")

```

```

25. pal_train_darker_rev = pal_train_darker.as_hex()[::-1]
26.
27. # %%
28. data_root = r'/home/drew/Desktop/MathModeling/APMCM24_cn/data'
29.
30. train_path = join(data_root, 'train.csv')
31. test_path = join(data_root, 'test.csv')
32.
33. train = pd.read_csv(train_path, index_col='id')
34. test = pd.read_csv(test_path, index_col='id')
35.
36. initial_features = list(test.columns)[: -1]
37.
38. train.head()
39.
40. # %% [markdown]
41. # # Problem 2
42. #
43. # 将附件 train.csv 中洪水发生的概率聚类成不同类别，分析具有高、中、低风险的
    洪水事件的指标特征。然后，选取合适的指标，计算不同指标的权重，建立发生洪水不
    同风险的预警评价模型，最后进行模型的灵敏度分析。
44. #
45. # 首先分析数据的可聚类性质，然后进行聚类分析。
46.
47. # %%
48. X = train.洪水概率.values.reshape(-1, 1)
49.
50. scaler = MinMaxScaler()
51. X_scaled = scaler.fit_transform(X)
52.
53. k = 3
54. kmeans = KMeans(
55.     n_clusters=k,
56.     verbose=2,
57.     random_state=114514
58. )
59.
60. kmeans.fit(X_scaled)
61. predicted_labels = kmeans.labels_
62. cluster_centers = kmeans.cluster_centers_
63.
64. # %%
65. # silhouette_avg = silhouette_score(X_scaled, predicted_labels)
66. # print("k-means 平均轮廓系数: ", silhouette_avg)

```

```

67.
68. # %%
69. train['聚类'] = predicted_labels
70.
71. plt.figure(figsize=(10, 6), dpi=300)
72. sns.boxplot(
73.     x='聚类', y='洪水概率',
74.     data=train,
75.     palette=pal_train_rev)
76. plt.title('K-Means++ 聚类结果箱形图')
77. plt.show()
78.
79. plt.figure(figsize=(12, 6), dpi=300)
80. sns.histplot(
81.     data=train,
82.     x='洪水概率',
83.     hue='聚类',
84.     bins=np.linspace(0.2825, 0.7275, 90),
85.     kde=True)
86. plt.ylabel('频数')
87. plt.title('K-Means++ 聚类结果直方图')
88. plt.show()
89.
90. # %%
91. train.head(10)
92.
93. # %%
94. # train.to_csv(join(data_root, 'train_cluster.csv'))
95.
96. # %%
97. result_dir = r'/home/drew/Desktop/MathModeling/APMCM24_cn/results'
98.
99. # %%
100. cls_df = train.groupby('聚类')
101. cls_0 = cls_df.get_group(0)
102. cls_1 = cls_df.get_group(1)
103. cls_2 = cls_df.get_group(2)
104.
105. # %%
106. cls_0.describe()
107. cls_0.describe().to_csv(join(result_dir, 'cls_0.csv'))
108.
109. # %%
110. cls_1.describe()

```

```

111. cls_1.describe().to_csv(join(result_dir, 'cls_1.csv'))
112.
113. # %%
114. cls_2.describe()
115. cls_2.describe().to_csv(join(result_dir, 'cls_2.csv'))
116.
117. # %%
118. vals_des = [cls_x.describe().洪水概
    率 for cls_x in [cls_0, cls_1, cls_2]]
119. idxs = ['低风险', '中风险', '高风险']
120. stat_df = pd.DataFrame(vals_des, index=idxs)
121. stat_df.to_csv(join(result_dir, 'stat.csv'))
122. stat_df.head()
123.
124. # %% [markdown]
125. # 可见，低风险、中风险和高风险的洪水事件概率分布是无交错的，故无需对获得的
    标签进行进一步处理。
126. #
127. # 然而，对于评估分类结果，我们仍旧需要进一步处理。
128. #
129. # 此处是回归任务，回归得出一个概率数值。然而，对于风险评级的划分，我们需要
    得出 [0,1] 区间内的三个子区间， 此时的分割
    为 [[0.285, 0.47], [0.475, 0.535], [0.54, 0.725]]，这些区间中间是有空缺的，
    如何填补这个空缺实现完整的一个回归区间划分从而实现回归+三分类模型的较优划分
    呢？
130. #
131. # 我们对 train 数据的洪水概率进行统计，基于其分布情况，进行区间填补。
132.
133. # %%
134. fitting_set = train[['洪水概率', '聚类']]
135. fit_train_X, fit_test_X, fit_train_y, fit_test_y = train_test_spli
    t(
136.     fitting_set.洪水概率, fitting_set.聚类,
137.     train_size=0.9, test_size=0.1,
138.     random_state=114514)
139.
140. # %%
141. fit_train_X = fit_train_X.values.reshape(-1, 1)
142. fit_test_X = fit_test_X.values.reshape(-1, 1)
143.
144. # %%
145. # l1_ratio = 0.5 # L1 weight in the Elastic-Net regularization
146.
147. # # Set regularization parameter

```

```

148. # for i, C in enumerate((1, 0.1, 0.01)):
149. #     # Increase tolerance for short training time
150. #     clf_l1_LR = LogisticRegression(C=C, penalty="l1", tol=0.01,
151. #                                     solver="saga")
152. #     clf_l2_LR = LogisticRegression(C=C, penalty="l2", tol=0.01,
153. #                                     solver="saga")
154. #     clf_en_LR = LogisticRegression(
155. #         C=C, penalty="elasticnet", solver="saga", l1_ratio=l1_ratio, tol=0.01
156. #     )
157. #     clf_l1_LR.fit(fit_train_X, fit_train_y)
158. #     clf_l2_LR.fit(fit_train_X, fit_train_y)
159. #     clf_en_LR.fit(fit_train_X, fit_train_y)
160. #
161. #     coef_l1_LR = clf_l1_LR.coef_.ravel()
162. #     coef_l2_LR = clf_l2_LR.coef_.ravel()
163. #     coef_en_LR = clf_en_LR.coef_.ravel()
164. #
165. #     # coef_l1_LR contains zeros due to the
166. #     # L1 sparsity inducing norm
167. #
168. #     sparsity_l1_LR = np.mean(coef_l1_LR == 0) * 100
169. #     sparsity_l2_LR = np.mean(coef_l2_LR == 0) * 100
170. #     sparsity_en_LR = np.mean(coef_en_LR == 0) * 100
171. #
172. #     print(f"C={C:.2f}")
173. #     print(f"{'Sparsity with L1 penalty':<40} {sparsity_l1_LR:.2f}%")
174. #     print(f"{'Sparsity with Elastic-Net penalty':<40} {sparsity_en_LR:.2f}%")
175. #     print(f"{'Sparsity with L2 penalty':<40} {sparsity_l2_LR:.2f}%")
176. #     print(f"{'Score with L1 penalty':<40} {clf_l1_LR.score(fit_test_X, fit_test_y):.2f}")
177. #     print(f"{'Score with Elastic-Net penalty':<40} {clf_en_LR.score(fit_test_X, fit_test_y):.2f}")
178. #     print(f"{'Score with L2 penalty':<40} {clf_l2_LR.score(fit_test_X, fit_test_y):.2f}")
179. #
180. #
181. # 1. **C 参数**:

```



```

182. # - `C` 是正则化强度的倒数。在这个案例中, `C` 的值从 1.00 逐渐减小
    到 0.01, 随着 `C` 的减小, 正则化强度增加。
183. #
184. # 2. **稀疏性 (Sparsity)**:
185. # - 稀疏性是指模型系数中有多少比例是精确为零的。L1 正则化因其稀疏性而
    特别有用, 因为它可以将不重要的特征系数压缩至零。然而, 这里的输出显示所有三种
    正则化方法都没有产生稀疏模型 (稀疏性为 0.00%), 这意味着没有特征系数被压缩至
    零。
186. #
187. # 3. **准确率 (Score)**:
188. # - 分数是模型在测试集上的准确度。对于 `C=1.00` 和 `C=0.10`, 所有模型
    的准确度都是 1.00 或非常接近 1.00。当 `C` 减小到 0.01 时, L1 和 L2 正则化
    模型的准确度略有下降至 0.93, 而 Elastic-Net 模型的准确度下降到 0.93
    (与 L2 相同)。
189. #
190. # 4. **正则化类型**:
191. # - L1 正则化 (Lasso): 即使在 `C` 值较小的情况下, 也没有产生稀疏模型,
    这可能表明所有特征都对模型的预测有贡献, 或者模型可能需要更小的 `C` 值来观察到
    稀疏性。
192. # - Elastic-Net 正则化: 随着 `C` 的减小, Elastic-Net 模型的准确度下
    降比 L1 和 L2 更显著, 这可能是因为 Elastic-Net 结合了 L1 和 L2 正则化的特
    性, 但在这个案例中没有产生稀疏性。
193. # - L2 正则化 (Ridge): 即使在不同的 `C` 值下, L2 正则化模型的准确度
    保持不变, 这与 L2 正则化不会将系数压缩至零的特性一致。
194. #
195. # 这些结果表明, 在这些特定的参数设置下, 模型可能没有遇到过拟合的问题, 因此
    正则化没有导致特征系数变为零。然而, 随着正则化强度的增加 (`C` 值减小), 模型
    的准确度有所下降, 这可能是由于模型复杂度的降低。
196. #
197. # 在实践中, 选择最佳的 `C` 值和正则化类型通常需要通过交叉验证等模型选择方
    法来确定, 以找到在防止过拟合和保持模型准确度之间最佳的平衡点。
198.
199. # %%
200. l1_ratio = 0.5 # L1 weight in the Elastic-Net regularization
201. C=1
202. clf_l1_LR = LogisticRegression(C=C, penalty="l1", tol=0.01, solver
    ="saga", verbose=1)
203. clf_l1_LR.fit(fit_train_X, fit_train_y)
204.
205. # %%
206. coef_l1_LR = clf_l1_LR.coef_.ravel()
207. sparsity_l1_LR = np.mean(coef_l1_LR == 0) * 100
208. print(f"C={C:.2f}")

```

```

209. print(f'{'Sparsity with L1 penalty:':<40} {sparsity_l1_LR:.2f}%")

210. print(f'{'Score with L1 penalty:':<40} {clf_l1_LR.score(fit_test_X
    , fit_test_y):.2f}")
211.
212. # %%
213. len(train), len(test)
214.
215. # %%
216. sample = np.linspace(0.2825, 0.7275, 100*1048575).reshape(-1, 1)
217. detailed = clf_l1_LR.predict(sample)
218. pred_df = pd.DataFrame({
219.     '样本': sample.squeeze(),
220.     '分类': detailed
221. })
222. pred_df.head()
223.
224. # %%
225. grp_pred = pred_df.groupby('分类')
226. grp_0 = grp_pred.get_group(0)
227. grp_1 = grp_pred.get_group(1)
228. grp_2 = grp_pred.get_group(2)
229.
230. # %%
231. preds_des = [grp_x.describe().样
    本 for grp_x in [grp_0, grp_1, grp_2]]
232. idxs = ['低风险', '中风险', '高风险']
233. p_stat_df = pd.DataFrame(preds_des, index=idxs)
234. p_stat_df.head()
235.
236. # %%
237. low = [0, p_stat_df['max'].loc['低风险']]
238. mid = [p_stat_df['min'].loc['中风险'], p_stat_df['max'].loc['中风险
    ']]
239. high = [p_stat_df['min'].loc['高风险'], 1]
240. low, mid, high
241.
242. # %% [markdown]
243. # 此时找到两个基本连续的区间
244. #
245. # [0.282500, 0.472481], [0.472481, 0.537549], [0.537549, 0.727500]

246. #
247. # 原始: [0.285, 0.47], [0.475, 0.535], [0.54, 0.725]

```

```

248. #
249. # 因此，得出区间划分的结论为：
250. #
251. # - 低风险区间：[0, 0.4724814817250219]
252. # - 中风险区间：[0.4724814859688767, 0.5375491376873294]
253. # - 高风险区间：[0.5375491419311842, 1]
254. #
255. # 此时，区间之间的差异很小，基本视为连续。可直接取平均数为分割点而不对结果
    产生影响
256.
257. # %%
258. l_m_sep = (p_stat_df['max'].loc['低风险'] + p_stat_df['min'].loc['
    中风险']) / 2
259. m_h_sep = (p_stat_df['max'].loc['中风险'] + p_stat_df['min'].loc['
    高风险']) / 2
260. low = [0, l_m_sep]
261. mid = [l_m_sep, m_h_sep]
262. high = [m_h_sep, 1]
263. low, mid, high
264.
265. # %%
266. cls_val_dist = pd.DataFrame(
267.     [cls_x.describe().loc['mean', :] for cls_x in [cls_0, cls_1, c
    ls_2]],
268.     columns=cls_0.columns,
269.     index=['低风险', '中风险', '高风险'])
270. ).drop(['洪水概率', '聚类'], axis=1).T
271. cls_val_dist.head()
272.
273. # %%
274. plt.figure(figsize=(14, 8), dpi=300) # 设置图形大小
275. sns.barplot(
276.     data=cls_val_dist,
277.     x=cls_val_dist.index, y='高风险',
278.     label='高风险', color='coral'
279. )
280. sns.barplot(
281.     data=cls_val_dist,
282.     x=cls_val_dist.index, y='中风险',
283.     label='中风险', color='teal'
284. )
285. sns.barplot(
286.     data=cls_val_dist,
287.     x=cls_val_dist.index, y='低风险',

```

```

288.         label='低风险', color='slategray'
289.     )
290.
291.     plt.legend()
292.
293.     plt.title('指标平均值在高中低风险下的分布情况')
294.     plt.xlabel('指标')
295.     plt.ylabel('指标平均值')
296.     plt.xticks(rotation=30)
297.
298.     plt.show()
299.
300.     # %% [markdown]
301.     # 回归模型建模
302.
303.     # %%
304.     import pandas as pd
305.     import numpy as np
306.     import matplotlib.pyplot as plt
307.     from matplotlib.ticker import MaxNLocator
308.     import seaborn as sns
309.     from colorama import Fore, Style
310.     import xgboost
311.     import lightgbm
312.     import catboost
313.     import os
314.     import datetime
315.     import pickle
316.     import warnings
317.     import umap
318.     import statsmodels.api as sm
319.
320.     from sklearn.base import clone
321.     from sklearn.model_selection import KFold, StratifiedKFold
322.     from sklearn.pipeline import make_pipeline, Pipeline
323.     from sklearn.decomposition import PCA
324.     from sklearn.preprocessing import StandardScaler, PolynomialFeatures, SplineTransformer, OneHotEncoder
325.     from sklearn.kernel_approximation import Nystroem
326.     from sklearn.linear_model import Ridge, LinearRegression
327.     from sklearn.ensemble import ExtraTreesRegressor, RandomForestRegressor
328.     from sklearn.metrics import r2_score
329.     from sklearn.manifold import TSNE

```

```

330. from sklearn.compose import TransformedTargetRegressor
331.
332. # %%
333. # Configuration
334. # Produce a submission file (you can set this to false if you only
335. # want to see the cross-validation results)
336. COMPUTE_TEST_PRED = True
337.
338. # Containers for results
339. oof, test_pred = {}, {}
340.
341. # %%
342. kf = KFold(n_splits=5, shuffle=True, random_state=1)
343.
344. SINGLE_FOLD = False
345.
346. weight_dir = r'/home/drew/Desktop/MathModeling/APMCM24_cn/weights'
347.
348. from joblib import dump
349.
350. def _classify(O_reg, label_cls, low, mid, high):
351.     cnt = 0
352.     for _, (o_reg, l_cls) in enumerate(zip(O_reg, label_cls)):
353.         o_cls = None
354.         if o_reg < low[1]:
355.             o_cls = 0
356.         elif o_reg >= mid[0] and o_reg < mid[1]:
357.             o_cls = 1
358.         elif o_reg >= high[0] and o_reg <= high[1]:
359.             o_cls = 2
360.         if o_cls == l_cls:
361.             cnt += 1
362.     return cnt
363.
364. def cross_validate(model, label, features=initial_features, n_repe
    ats=1):
365.     """Compute out-of-fold and test predictions for a given model.
366.
367.     Out-of-fold and test predictions are stored in the global vari
    ables
368.     oof and test_pred, respectively.

```

```

369.
370.     If n_repeats > 1, the model is trained several times with different seeds.
371.     """
372.     best_model = None
373.     best_score = float('-inf')
374.
375.     start_time = datetime.datetime.now()
376.     scores = []
377.     oof_preds = np.full_like(train.洪水概率, np.nan, dtype=float)
378.     cls_cnt, sample_num = 0, 0
379.     for fold, (idx_tr, idx_va) in enumerate(kf.split(train)):
380.         X_train = train.iloc[idx_tr][features]
381.         X_val = train.iloc[idx_va][features]
382.         y_train = train.iloc[idx_tr].洪水概率
383.         y_val = train.iloc[idx_va].洪水概率
384.         y_label = train.iloc[idx_va].聚类
385.
386.         y_pred = np.zeros_like(y_val, dtype=float)
387.         for i in range(n_repeats):
388.             m = clone(model)
389.             if n_repeats > 1:
390.                 mm = m
391.                 if isinstance(mm, Pipeline):
392.                     mm = mm[-1]
393.                 mm.set_params(random_state=i)
394.                 m.fit(X_train, y_train)
395.                 dump(m, os.path.join(weight_dir, f'{label}_{fold}_{i}.joblib'))
396.                 y_pred += m.predict(X_val)
397.                 y_pred /= n_repeats
398.                 sample_num += len(y_label)
399.
400.                 cls_cnt += _classify(y_pred, y_label, low, mid, high)
401.                 acc_3 = cls_cnt / sample_num
402.                 score = r2_score(y_val, y_pred)
403.                 print(f"# Fold {fold}: R2={score:.5f}, acc_3={acc_3:.5f}")
404.
405.                 if score > best_score:
406.                     best_score = score
407.                     best_model = clone(m) # 克隆当前模型实例
408.
409.         scores.append(score)

```

```

410.         oof_preds[idx_va] = y_pred
411.         if SINGLE_FOLD: break
412.
413.         elapsed_time = datetime.datetime.now() - start_time
414.         print(f"{Fore.GREEN}# Overall: {np.array(scores).mean():.5f} {
label}")
415.         f"{' single fold' if SINGLE_FOLD else ''}"
416.         f" {int(np.round(elapsed_time.total_seconds() / 60))}
min{Style.RESET_ALL}")
417.         oof[label] = oof_preds
418.
419.         if COMPUTE_TEST_PRED:
420.             # Retrain n_repeats times with the whole dataset and avera
ge
421.             y_pred = np.zeros(len(test), dtype=float)
422.             X_train = train[features]
423.             y_train = train.洪水概率
424.             for i in range(n_repeats):
425.                 m = clone(model)
426.                 if n_repeats > 1:
427.                     mm = m
428.                     if isinstance(mm, Pipeline):
429.                         mm = mm[-1]
430.                     if isinstance(mm, TransformedTargetRegressor):
431.                         mm = mm.regressor
432.                         mm.set_params(random_state=i)
433.                         m.fit(X_train, y_train)
434.                         y_pred += m.predict(test[features])
435.             y_pred /= n_repeats
436.             test_pred[label] = y_pred
437.
438.         return best_model
439.
440.     # %%
441.     X = sm.add_constant(train[initial_features])
442.     res = sm.OLS(train.洪水概率, X, missing='error').fit()
443.     res.summary()
444.
445.     # %%
446.     res_summary = res.summary()
447.     res_summary.tables[0]
448.
449.     # %%
450.     res_summary.tables[1]

```

```

451.
452. # %%
453. res_summary.tables[2]
454.
455. # %%
456. res_df_list = [pd.DataFrame(table) for table in res_summary.tables
457.                 ]
458. for idx, df in enumerate(res_df_list):
459.     df.to_csv(join(result_dir, f'ols_{idx}.csv'))
460.
461. # %% [markdown]
462. # 根据您提供的普通最小二乘法（OLS）回归分析结果，我们可以对模型进行以下分
463. # 析：
464. #
465. # 1. **模型概述**：
466. #     - 因变量（Dep. Variable）是“洪水概率”，即预测洪水发生的概率。
467. #     - R-squared（决定系数）为 0.792，表明模型解释了 79.2% 的变量变异性，
468. #       这是一个相对较高的值，说明模型拟合效果较好。
469. #     - Adj. R-squared（调整后的决定系数）也为 0.792，这表明即使在添加了
470. #       多个解释变量后，模型的解释能力仍然很高。
471. #
472. # 2. **模型统计检验**：
473. #     - F-statistic（F 统计量）为 2.097e+05，这是一个非常高的值，表明模型
474. #       中至少有一个解释变量对因变量有显著影响。
475. #     - Prob（F-statistic）（F 统计量的 P 值）为 0.00，远小于 0.05，表明模
476. #       型在统计上是显著的。
477. #
478. # 3. **模型拟合优度**：
479. #     - Log-Likelihood（对数似然函数）为 2.4543e+06，用于模型比较。
480. #     - AIC（赤池信息准则）和 BIC（贝叶斯信息准则）分别为 -4.909e+06 和
481. #       -4.908e+06，这两个值通常用于模型选择，值越低表示模型拟合得越好。
482. #
483. # 4. **模型参数估计**：
484. #     - 回归系数（coef）显示了每个解释变量对“洪水概率”的影响大小。所有特征
485. #       的系数都非常接近 0.0056，这表明每个特征的增加都会使洪水概率增加 0.56%（在保持
486. #       其他变量不变的情况下）。
487. #     - 标准误差（std err）非常小，表明估计的系数非常精确。
488. #     - t 统计量（t）和 P>|t|（t 统计量的 P 值）都表明所有特征在统计上都是显
489. #       著的。
490. #
491. # 5. **模型诊断**：
492. #     - Omnibus（总体检验）和 Prob(Omnibus）（总体检验的 P 值）用于检测残差
493. #       的正态性，这里的 P 值为 0.000，表明残差可能不服从正态分布。

```



```

483. # - Durbin-Watson (德宾-沃森检验) 值为 2.001, 接近 2, 表明没有自相关性。
484. # - Jarque-Bera (JB) (贾奎-贝拉检验) 的 P 值为 0.00, 表明残差的分布可能偏离正态分布。
485. # - Skewness (偏度) 为 0.402, Kurtosis (峰度) 为 3.428, 这些值表明残差分布可能不对称, 并且有厚尾。
486. #
487. # 6. **模型稳健性**:
488. # - Covariance Type (协方差类型) 为 nonrobust, 表明模型的系数估计可能对异常值敏感。
489. #
490. # 7. **条件数**:
491. # - Cond. No. (条件数) 为 250, 这个值相对较低, 表明模型没有严重的多重共线性问题。
492. #
493. # 综上所述, 您的 OLS 回归模型在统计上是显著的, 拟合效果较好, 但残差的分布可能不完全符合正态分布的假设。在实际应用中, 可能需要进一步的模型诊断和改进。
494. #
495. # 所有特征的系数基本相同, 都是 0.0056。我们可以用散文的形式解释这个模型:
496. #
497. # > **将所有特征的总和乘以 0.0056, 再减去 0.05, 你就可以得到洪水发生的概率。
    **
498. #
499. # 我们可以用一行代码实现这一点, 并得到 0.844 的 R 平方分数:
500. #
501. # > **"将所有特征的总和乘以 0.0056, 然后减去 0.05, 就可以得到洪水发生的概率。" 在这里, R 平方分数 (R-squared score) 0.844 表明该模型能解释洪水概率变异的 84.4%。
502. #
503.
504. # %%
505. X_0 = sm.add_constant(cls_0[initial_features])
506. res_0 = sm.OLS(cls_0.洪水概率, X_0, missing='error').fit()
507. res_0.summary()
508.
509. # %%
510. X_1 = sm.add_constant(cls_1[initial_features])
511. res_1 = sm.OLS(cls_1.洪水概率, X_1, missing='error').fit()
512. res_1.summary()
513.
514. # %%
515. X_2 = sm.add_constant(cls_2[initial_features])
516. res_2 = sm.OLS(cls_2.洪水概率, X_2, missing='error').fit()
517. res_2.summary()

```

```

518.
519. # %%
520. r2_tr = r2_score(train.洪水概
    率, (train[initial_features].sum(axis=1) * 0.0056) - 0.0181)
521. r2_0 = r2_score(cls_0.洪水概
    率, (cls_0[initial_features].sum(axis=1) * 0.0035) + 0.1427)
522. r2_1 = r2_score(cls_1.洪水概
    率, (cls_1[initial_features].sum(axis=1) * 0.0021) + 0.3076)
523. r2_2 = r2_score(cls_2.洪水概
    率, (cls_2[initial_features].sum(axis=1) * 0.0035) + 0.2051)
524. r2_tr, r2_0, r2_1, r2_2
525.
526. # %% [markdown]
527. # 基准 XGBoost
528.
529. # %%
530. # XGBoost
531. xgb_params = {'grow_policy': 'depthwise', 'n_estimators': 100, 'le
    arning_rate': 0.2639887908316703, 'max_depth': 10, 'reg_lambda': 62.4
    6661785864016, 'min_child_weight': 0.33652299514909034, 'colsample_by
    tree': 0.2319730052165745, 'objective': 'reg:squarederror', 'tree_met
    hod': 'hist', 'max_bin': 2048, 'gamma': 0} # 0.83868
532. model = xgboost.XGBRegressor(**xgb_params)
533. cross_validate(model, 'XGBoost')
534. # Overall: 0.83949 XGB single fold 0 min
535.
536. # %% [markdown]
537. # 基准 CatBoost 模型
538.
539. # %%
540. model = catboost.CatBoostRegressor(verbose=False)
541. cross_validate(model, 'CatBoost')
542. # Overall: 0.84722 CB single fold 1 min
543.
544. # %% [markdown]
545. # 基准模型的性能并不尽如人意，对数据进行深入观察
546.
547. # %%
548. temp = train.洪水概
    率.groupby(train[initial_features].sum(axis=1)).mean()
549. plt.figure(dpi=300)
550. plt.scatter(temp.index, temp, marker='o', s=2, cmap='PuBuGn')
551. plt.scatter(temp.index, temp, marker='^',
552.             s=4, c=(temp.index.isin(np.arange(62, 74))),

```

```

553.             cmap='coolwarm', )
554. plt.xlabel('20 指标(特征)之和')
555. plt.ylabel('平均洪水概率')
556. plt.grid()
557. plt.show()
558.
559. # %% [markdown]
560. # 可以观察到，洪水发生概率与特征之和基本呈正相关，同时，观察到线性关系中存在特殊值，因此，我们将两者都纳入模型考虑范围
561.
562. # %%
563. for df in [train, test]:
564.     df['指标和'] = df[initial_features].sum(axis=1)
565.     df['特殊点'] = df['指标和'].isin(np.arange(62, 74))
566.
567. # %%
568. train.head()
569.
570. # %%
571. # model = catboost.CatBoostRegressor(verbose=False)
572. # cross_validate(model, 'CatBoost fsum', features=initial_features
    +['指标和'])
573. # # Overall: 0.86840 CatBoost single fold 2 min
574.
575. # %% [markdown]
576. # 将指标进行从小到大排序
577.
578. # %%
579. sorted_features = [f"指标排序
    {i}" for i in np.arange(len(initial_features))]
580. for df in [train, test]:
581.     df[sorted_features] = np.sort(df[initial_features], axis=1)
582.     train.head()
583.
584. # %%
585. # model = catboost.CatBoostRegressor(verbose=False)
586. # cross_validate(model, 'CatBoost sorted', features=sorted_features+['指标和'])
587.
588. # %%
589. for df in [train, test]:
590.     df['中位数'] = np.median(df[initial_features], axis=1)
591.     df['标准差'] = np.std(df[initial_features], axis=1)
592.     df['平均数'] = np.mean(df[initial_features], axis=1)

```

```

593.
594. # %%
595. import itertools
596.
597. extra_features = [
598.     '中位数',
599.     '标准差',
600.     '平均数',
601. ]
602.
603. power_set = list(itertools.chain(
604.     *[itertools.combinations(extra_features, r)
605.       for r in range(len(extra_features) + 1)]))
606.
607. power_set_as_lists = [list(subset) for subset in power_set]
608.
609. print(power_set_as_lists)
610.
611. # %% [markdown]
612. # 构造幂集进行多方测试
613.
614. # %%
615. # for idx, ef in enumerate(power_set_as_lists):
616. #     print(idx + 1, ef)
617. #     model = catboost.CatBoostRegressor(verbose=False)
618. #     cross_validate(model, 'CatBoost extra', features=sorted_features+['指标和']+ef)
619.
620. # 1 []
621. # # Fold 0: R2=0.80326, acc_3=0.73407
622. # # Overall: 0.80326 CatBoost extra single fold 1 min
623. # 2 ['中位数']
624. # # Fold 0: R2=0.80322, acc_3=0.73376
625. # # Overall: 0.80322 CatBoost extra single fold 1 min
626. # 3 ['标准差']
627. # # Fold 0: R2=0.80317, acc_3=0.73381
628. # # Overall: 0.80317 CatBoost extra single fold 1 min
629. # 4 ['平均数']
630. # # Fold 0: R2=0.80323, acc_3=0.73409
631. # # Overall: 0.80323 CatBoost extra single fold 1 min
632. # 5 ['中位数', '标准差']
633. # # Fold 0: R2=0.80317, acc_3=0.73387
634. # # Overall: 0.80317 CatBoost extra single fold 1 min
635. # 6 ['中位数', '平均数']

```

```

636. # # Fold 0: R2=0.80324, acc_3=0.73437
637. # # Overall: 0.80324 CatBoost extra single fold 1 min
638. # 7 ['标准差', '平均数']
639. # # Fold 0: R2=0.80323, acc_3=0.73420
640. # # Overall: 0.80323 CatBoost extra single fold 1 min
641. # 8 ['中位数', '标准差', '平均数']
642. # # Fold 0: R2=0.80320, acc_3=0.73423
643. # # Overall: 0.80320 CatBoost extra single fold 1 min
644.
645. # %% [markdown]
646. # 发现最优情况是不加入额外特征
647.
648. # %%
649. best = catboost.CatBoostRegressor(
650.     verbose=False,
651. )
652. best_model = cross_validate(
653.     best,
654.     'CatBoost_FeatureSum_Sorted',
655.     features=sorted_features+['指标和'],
656.     n_repeats=4
657. )
658.
659. # %%
660. result_df = pd.read_csv(join(data_root, 'submit.csv'), index_col=0
661. )
662. result_df.head()
663.
664. # %%
665. if COMPUTE_TEST_PRED:
666.     result_df['洪水概率
667.         '] = test_pred['CatBoost_FeatureSum_Sorted']
668.     result_df.head(10)
669.
670. # %%
671. filename = 'submit.csv'
672. result_df.to_csv(filename)
673. os.system(f"head {filename}")
674.
675. # %%
676. flood_df = result_df.洪水概率
677. # 计算偏度和峰度
678. flood_df_skew = stats.skew(flood_df.dropna()) # 删除 NaN 值并计算偏
        度

```

```

677. flood_df_kurt = stats.kurtosis(flood_df.dropna()) # 删除 NaN 值并
    计算峰度
678. plt.figure(figsize=(5, 2), dpi=300)
679. plt.hist(
680.     flood_df,
681.     bins=np.linspace(0.3,0.7,300),
682.     density=True,
683.     color='teal',
684. )
685. sns.kdeplot(
686.     flood_df,
687.     color='steelblue',
688.     shade=True,
689. )
690. # 添加偏度和峰度到图的左上角
691. plt.text(
692.     0.05, # x 位置
693.     0.95, # y 位置, 可以根据你的图形布局进行调整
694.     f'偏度: {flood_df_skew:.4f}\n峰度: {flood_df_kurt:.4f}', # 要
    显示的文本
695.     horizontalalignment='left', # 水平对齐方式
696.     verticalalignment='top', # 垂直对齐方式
697.     transform=plt.gca().transAxes, # 将位置指定为图形坐标轴的比例
698.     fontsize=8
699. )
700. plt.ylabel('概率密度')
701. plt.xlabel('洪水概率')
702. plt.show()
703.
704. # %%
705. flood_probabilities = result_df['洪水概率
    '].sample(1000).dropna() # 删除 NaN 值
706.
707. plt.figure(figsize=(18, 6),dpi=300)
708.
709. plt.plot(
710.     flood_probabilities.values,
711.     label='洪水概率',
712.     color='teal'
713. )
714. plt.title('洪水概率折线图')
715. plt.ylabel('洪水概率')
716. plt.xlabel('随机采样数据 (1000 条)')
717. plt.show()

```

```
718.  
719.  # %%  
720.  from joblib import load  
721.  
722.  name = 'CatBoost_FeatureSum_Sorted_0_3.joblib'  
723.  weight_path = join(weight_dir, name)  
724.  model = load(weight_path)  
725.  
726.  # %%  
727.  model.feature_importances_, model.feature_names_  
728.  
729.  # %%  
730.  model.best_score_
```